



US012406381B2

(12) **United States Patent**
Deng et al.

(10) **Patent No.:** **US 12,406,381 B2**
(45) **Date of Patent:** **Sep. 2, 2025**

(54) **EDGE-BASED GEOMETRY
REPRESENTATION FOR CROSS LAYER
APPLICATIONS**

(71) Applicant: **QUALCOMM Incorporated**, San
Diego, CA (US)

(72) Inventors: **Yan Deng**, La Jolla, CA (US); **Michel
Adib Sarkis**, San Diego, CA (US);
Ning Bi, San Diego, CA (US); **Nikolai
Konrad Leung**, San Francisco, CA
(US)

(73) Assignee: **QUALCOMM Incorporated**, San
Diego, CA (US)

(*) Notice: Subject to any disclaimer, the term of this
patent is extended or adjusted under 35
U.S.C. 154(b) by 447 days.

(21) Appl. No.: **18/174,525**

(22) Filed: **Feb. 24, 2023**

(65) **Prior Publication Data**
US 2024/0289970 A1 Aug. 29, 2024

(51) **Int. Cl.**
G06T 7/12 (2017.01)
G06T 7/33 (2017.01)
(Continued)

(52) **U.S. Cl.**
CPC **G06T 7/344** (2017.01); **G06T 7/12**
(2017.01); **G06T 17/00** (2013.01); **G06V 20/70**
(2022.01);
(Continued)

(58) **Field of Classification Search**
CPC G06T 7/344; G06T 7/12; G06T 17/00;
G06T 2207/20021; G06T 2207/20221;
(Continued)

(56) **References Cited**

U.S. PATENT DOCUMENTS

2017/0358048 A1* 12/2017 Kotake G06T 1/0007
2018/0189956 A1* 7/2018 Mehr G06T 7/97
(Continued)

OTHER PUBLICATIONS

Blut C H., et al., "Mobile Augmented Reality for Semantic 3D
Models: A Smartphone-Based Approach with CityGML", RWTH
Aachen University, May 24, 2019, 299 Pages, XP093146267,
Aachen, Figures 2.50-2.52, p. 111, Paragraph 2—p. 115, Paragraph
1.

(Continued)

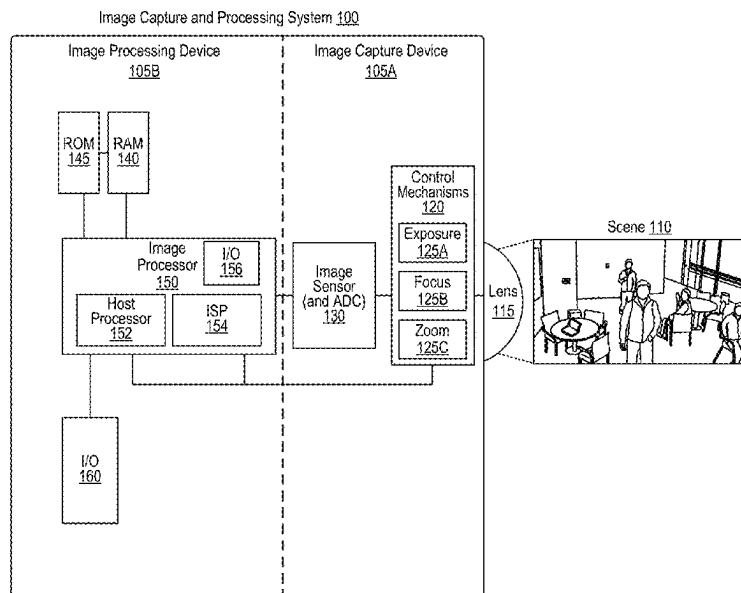
Primary Examiner — Jitesh Patel

(74) *Attorney, Agent, or Firm* — Polsinelli
LLP/QUALCOMM Incorporated

(57) **ABSTRACT**

Techniques and systems are provided for image processing.
For instance, a process can include obtaining, from one or
more image sensors, a first image of an environment;
determining semantic labels for a plurality of pixels of the
first image based on whether each pixel of the plurality of
pixels is associated with an object in the first image to
generate a semantically segmented image; applying a two-
dimensional line representation of the first image to the
semantically segmented image to generate a labeled 2D line
representation; back-projecting the labeled 2D line repre-
sentation to three dimensions to generate a line representa-
tion model; fusing the line representation model and the
semantically segmented image to generate a labeled line repre-
sentation model; and outputting the labeled line repre-
sentation model.

30 Claims, 11 Drawing Sheets



- (51) **Int. Cl.**
G06T 17/00 (2006.01)
G06V 20/70 (2022.01)
- (52) **U.S. Cl.**
 CPC *G06T 2207/20021* (2013.01); *G06T 2207/20221* (2013.01)
- (58) **Field of Classification Search**
 CPC G06T 7/33; G06V 20/70; G06V 10/94;
 G06V 10/44; G06V 10/82; G06V 20/64;
 G06V 10/46; G06V 20/20
 See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

2021/0264635 A1* 8/2021 Chen G06T 7/33
 2024/0125899 A1* 4/2024 Khadem G06V 10/82

OTHER PUBLICATIONS

Chen L., et al., "Context-Aware Mixed Reality: A Learning-Based Framework for Semantic-Level Interaction", Computer Graphics Forum : Journal of the European Association for Computer Graph-

ics, vol. 39, No. 1, Feb. 1, 2020, pp. 484-496, XP055707662, Oxford, Abstract, Figures 1-4, 6, 7, p. 487, col. 2, Paragraph 1—p. 491, col. 2, Paragraph 3.
 International Search Report and Written Opinion—PCT/US2024/011051—ISA/EPO—Apr. 10, 2024.
 Lianos K.-N., et al., "Robust Planar Optimization for General 3D Room Layout Estimation", 2022 IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct), IEEE, Oct. 17, 2022, pp. 875-880, XP034248195, Abstract, Figures 1-3, 7, 9, p. 876, col. 2, Paragraph 2—p. 878, col. 2, Paragraph 3.
 Lima J.P., et al., "Standalone Edge-Based Markerless Tracking of Fully 3-Dimensional Objects for Handheld Augmented Reality", Proceedings of the 16th ACM Symposium on Virtual Reality Software and Technology, Nov. 18-20, 2009, Kyoto, Japan, ACM, 2 Penn Plaza, Suite 701 New York NY 10121-0701 USA, Nov. 18, 2009, pp. 139-142, XP058210125, Abstract, Figures 2, 5, p. 139, col. 2, Paragraph 3—p. 142, col. 1, Line 5, Abstract, Figures 1, 2, 5, p. 139, col. 2, Paragraph 3—p. 142, Line 5.
 Schutt P., et al., "Semantic Interaction in Augmented Reality Environments for Microsoft HoloLens", 2019 European Conference on Mobile Robots (ECMR), IEEE, Sep. 4, 2019, 6 Pages, XP033635181, Abstract, Figures 1, 2, 5 p. 2, col. 2, Paragraph 1—p. 5, col. 2, Paragraph 1, Abstract, Figures 1, 2, 4, 5, p. 2, col. 2, Paragraph 1—p. 4, col. 2, Paragraph 2.

* cited by examiner

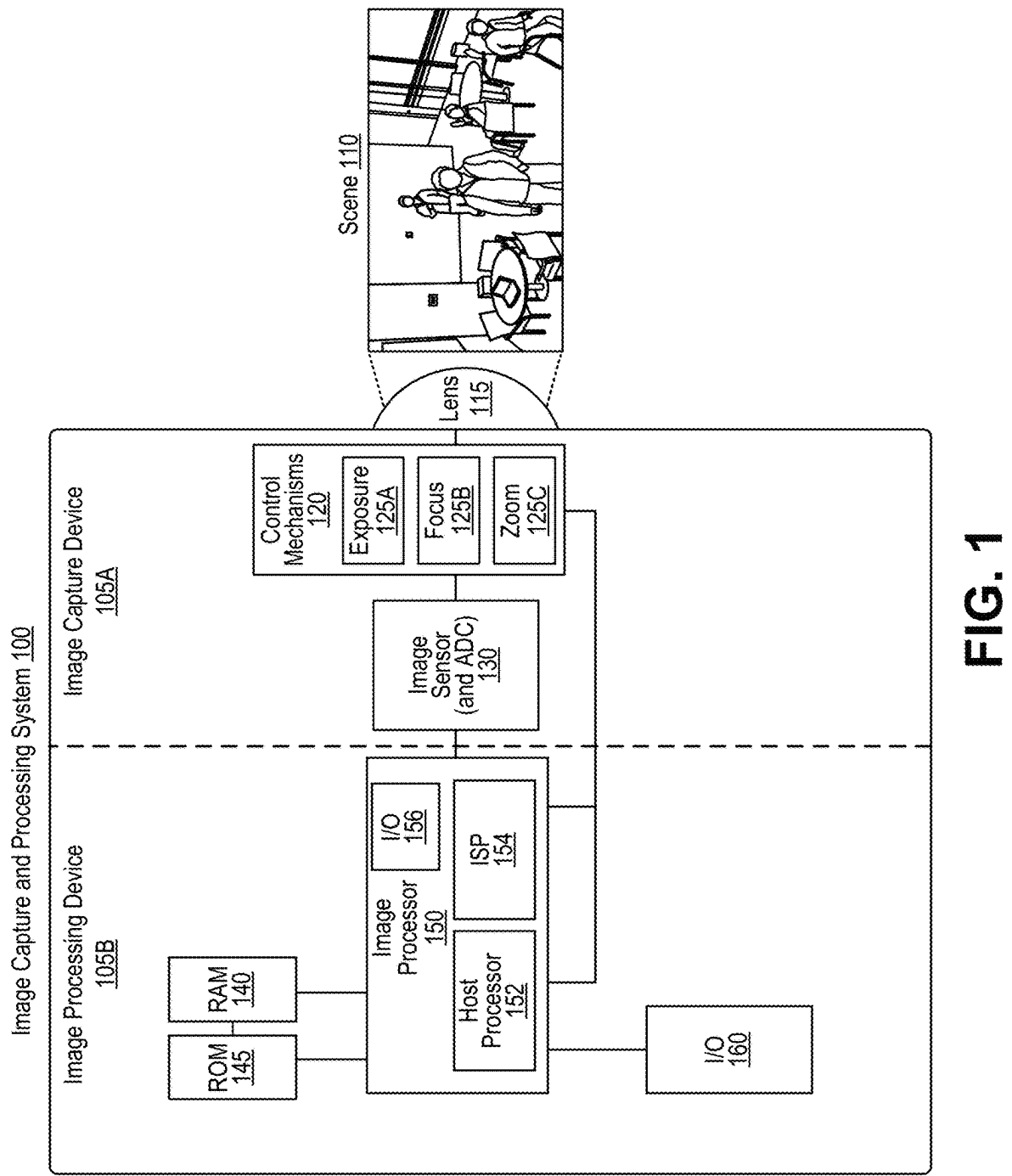


FIG. 1

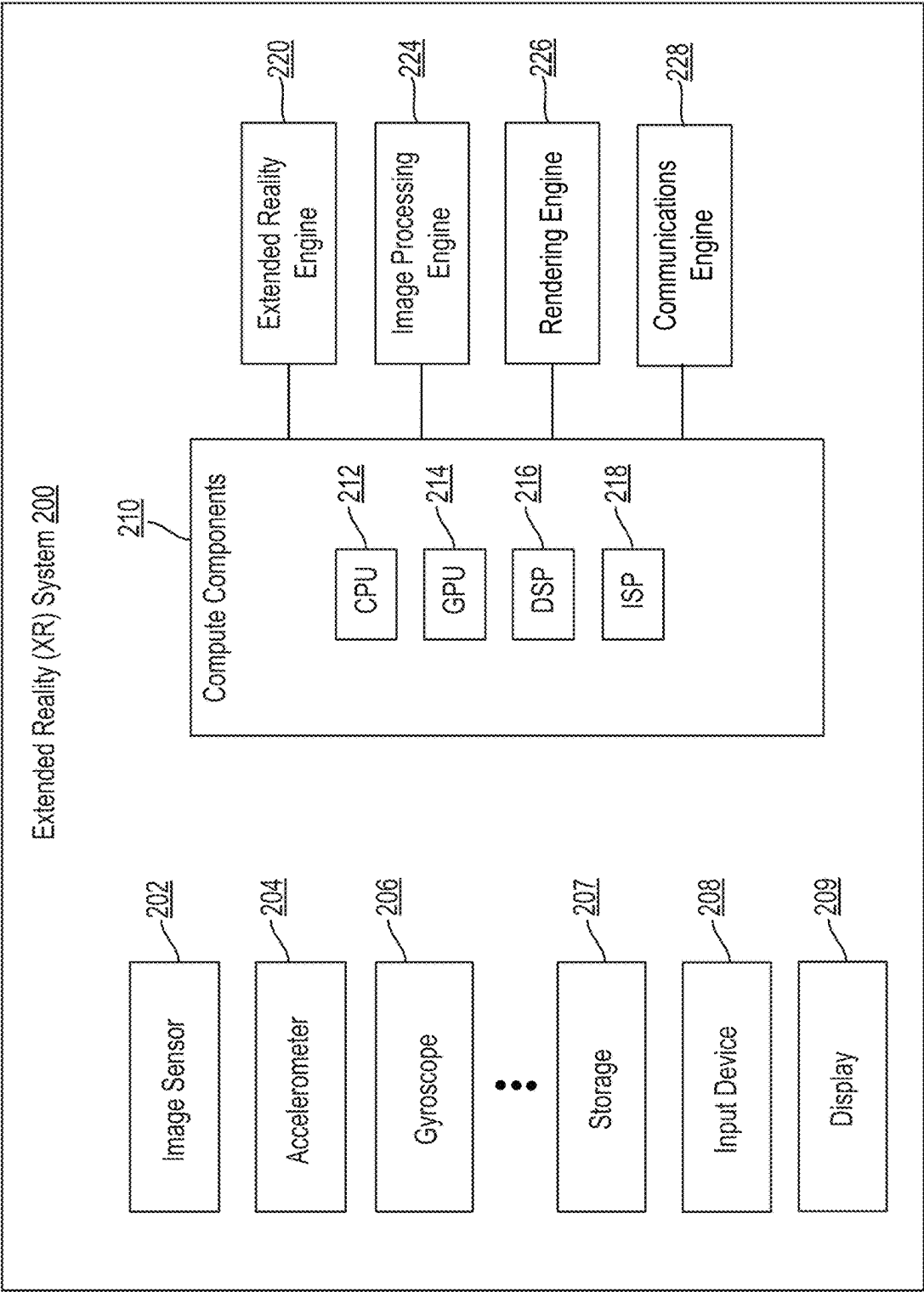


FIG. 2

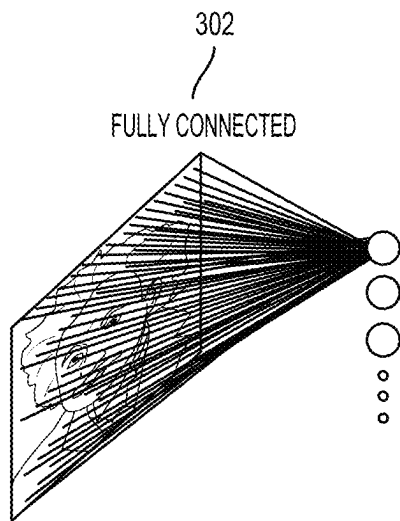


FIG. 3A

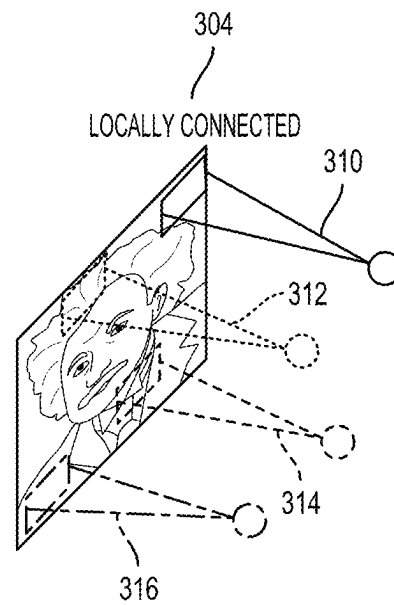


FIG. 3B

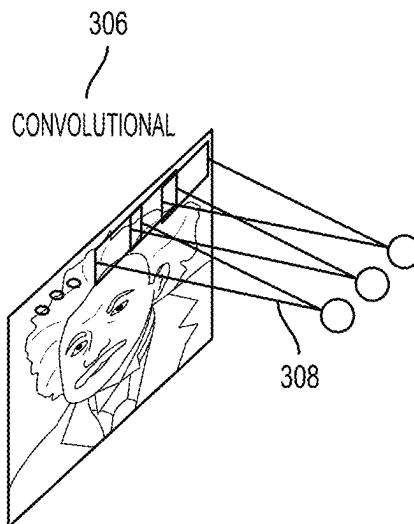


FIG. 3C

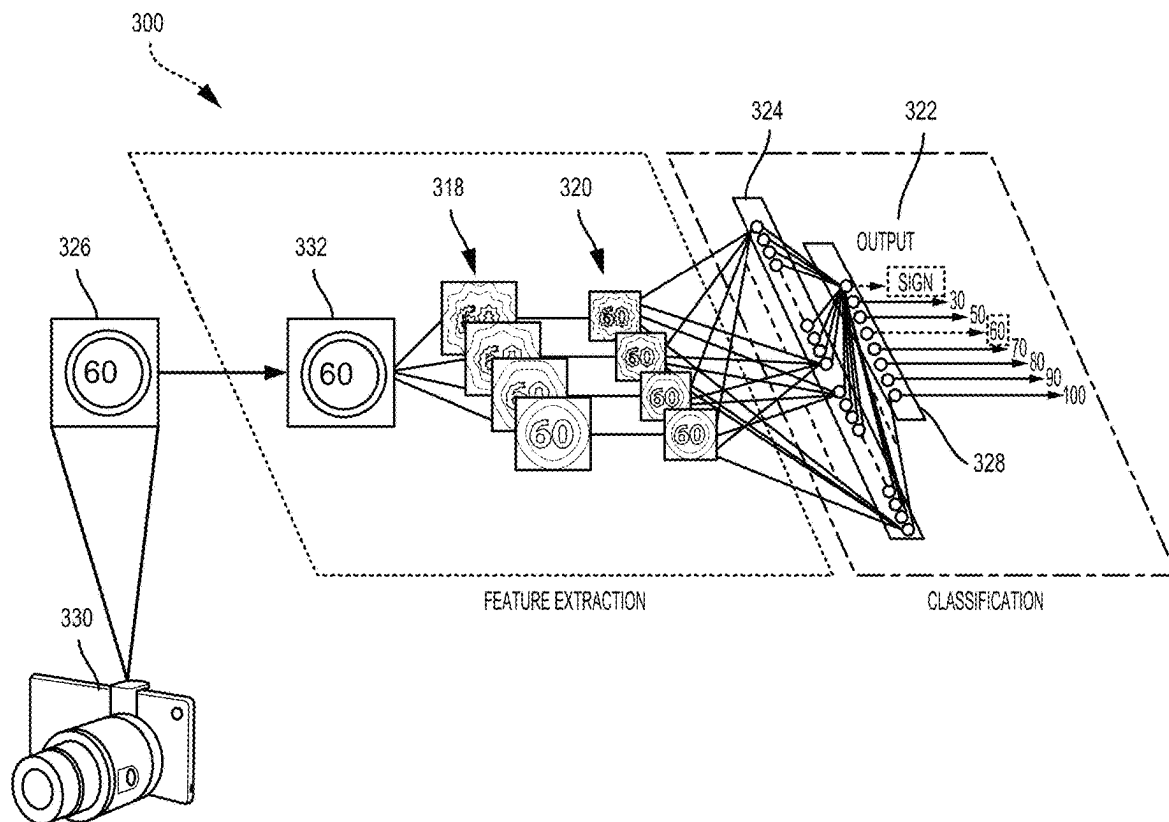
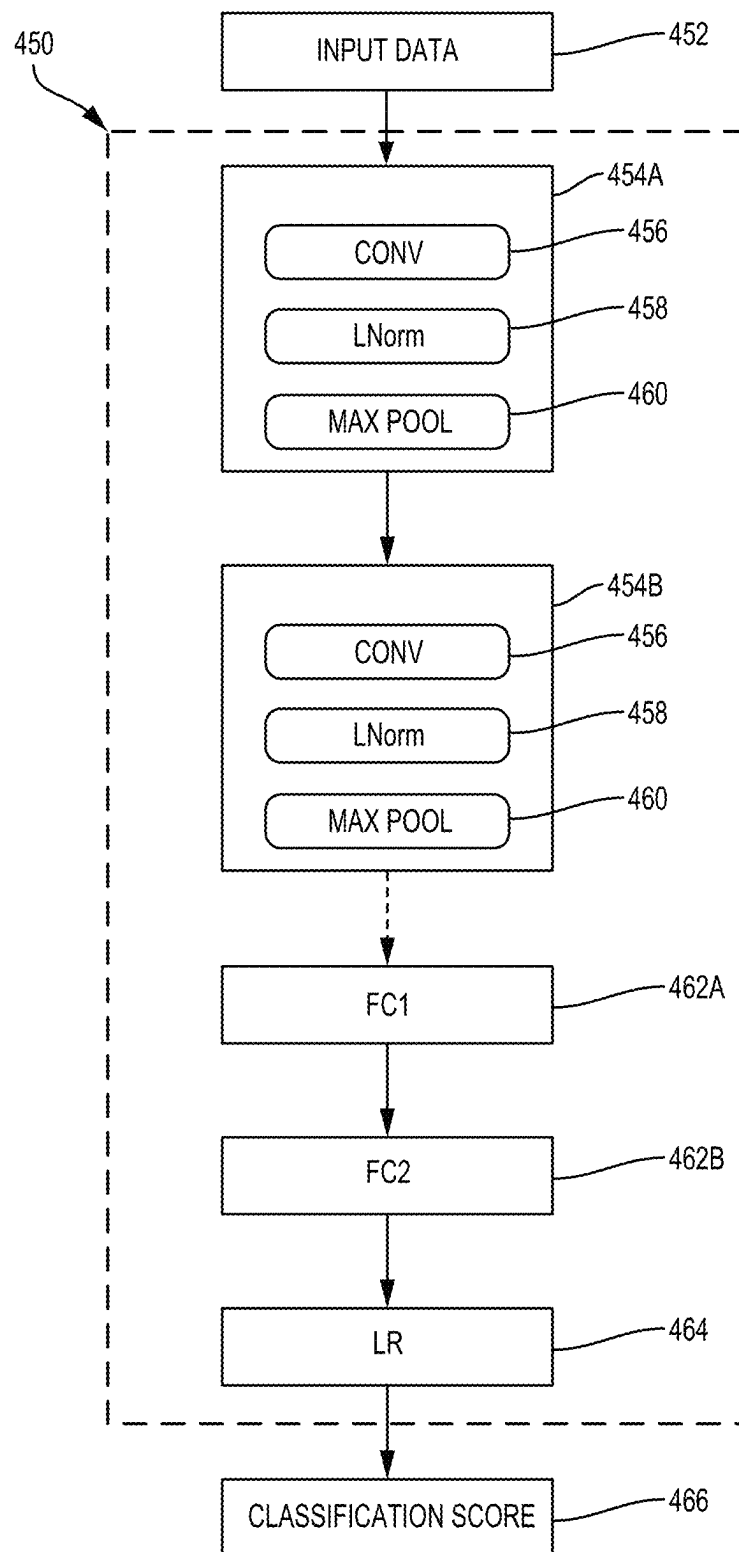


FIG. 3D

**FIG. 4**

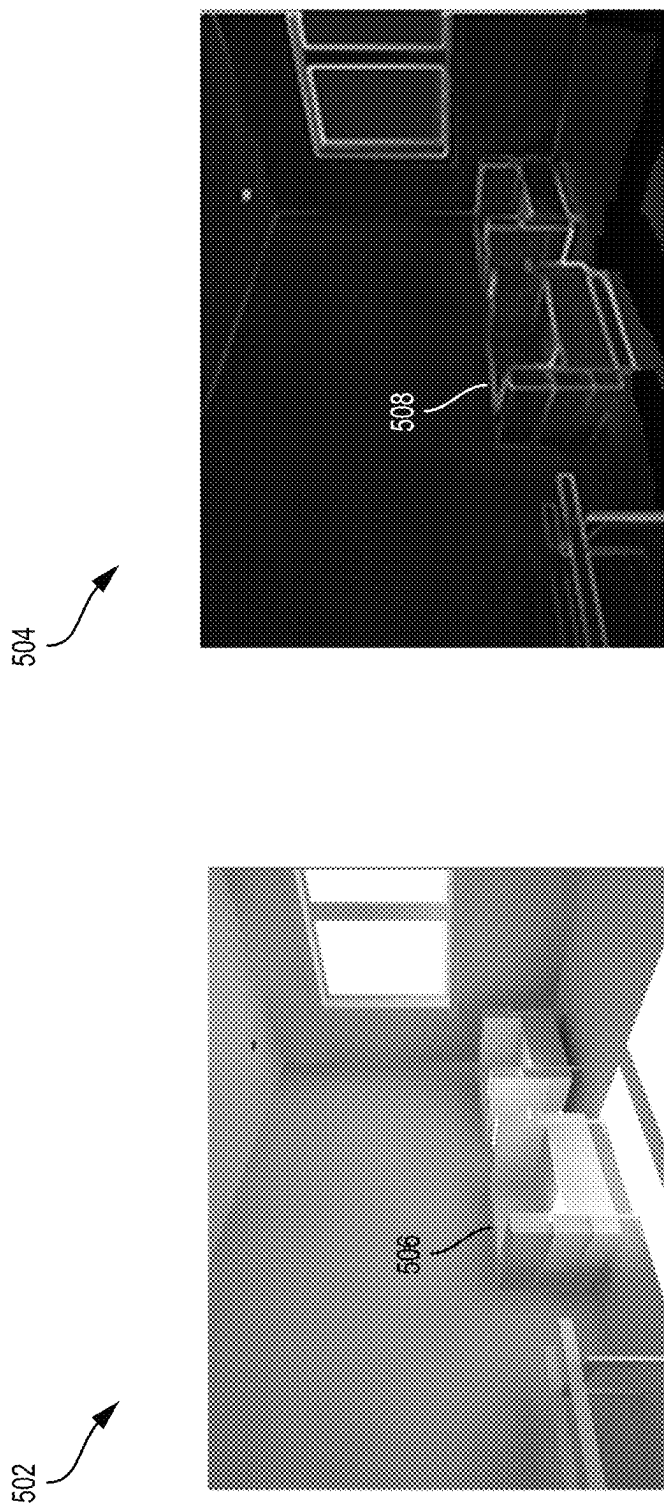


FIG. 5

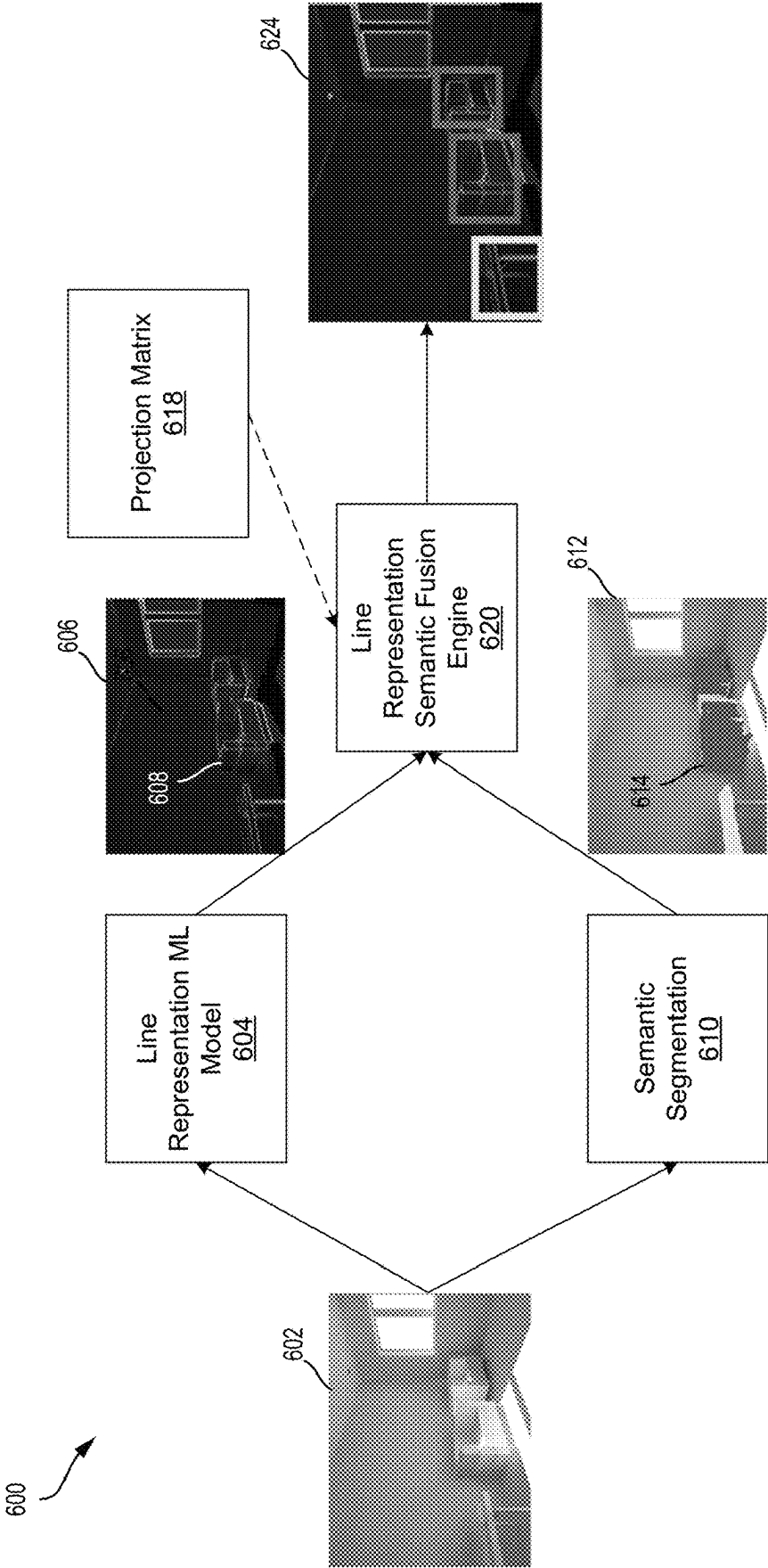
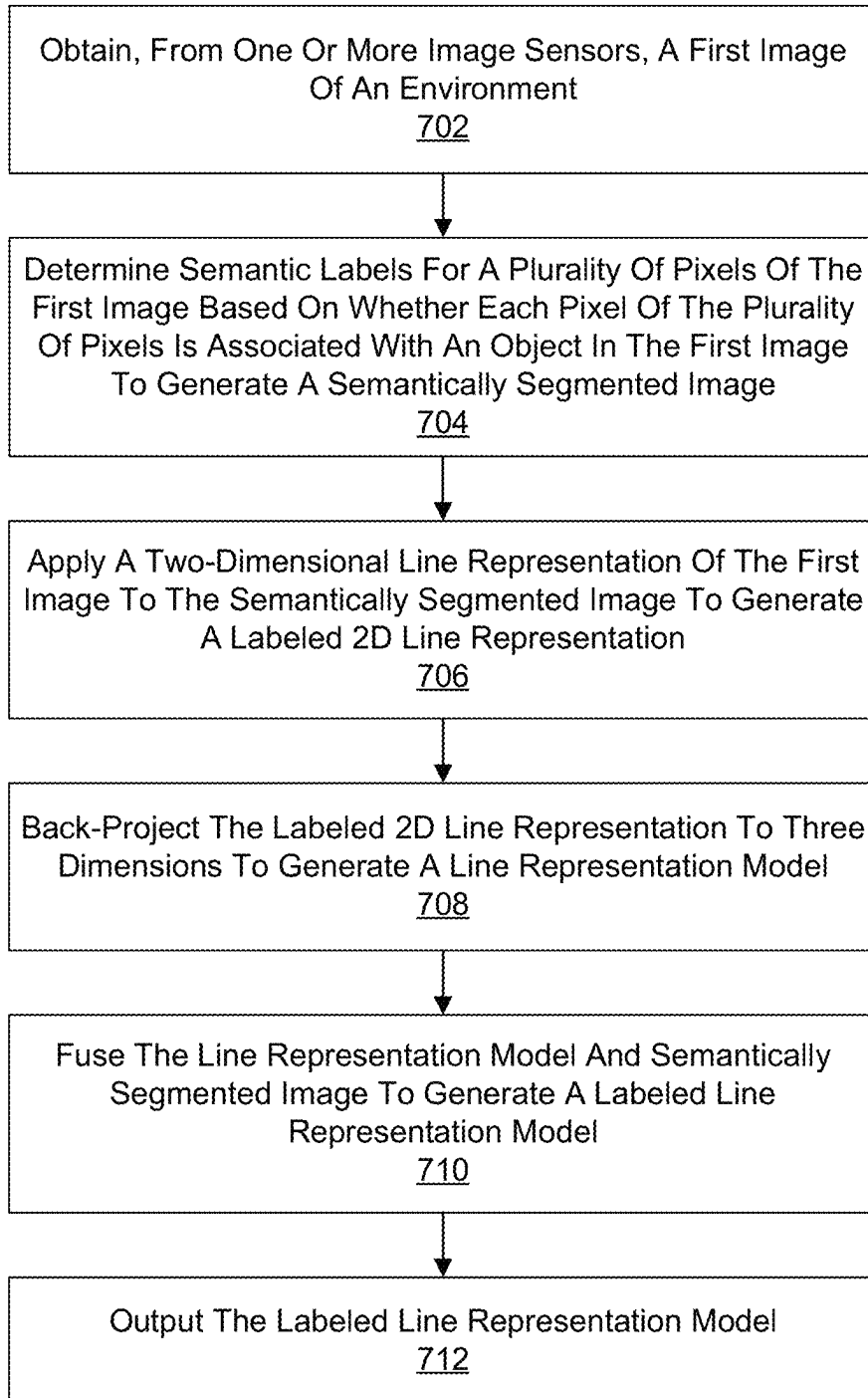


FIG. 6

700

**FIG. 7**

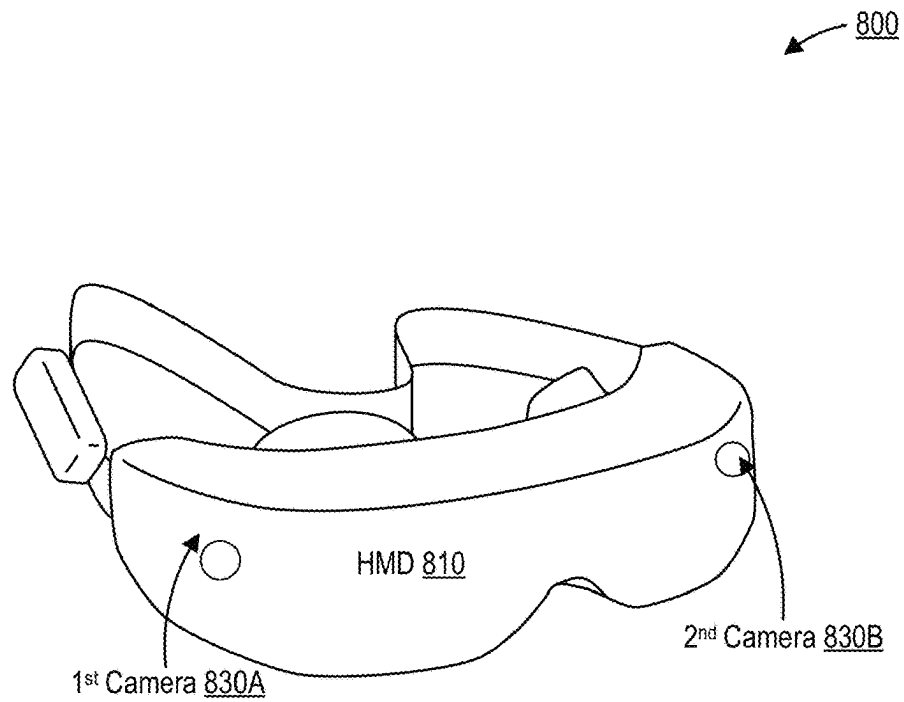


FIG. 8A

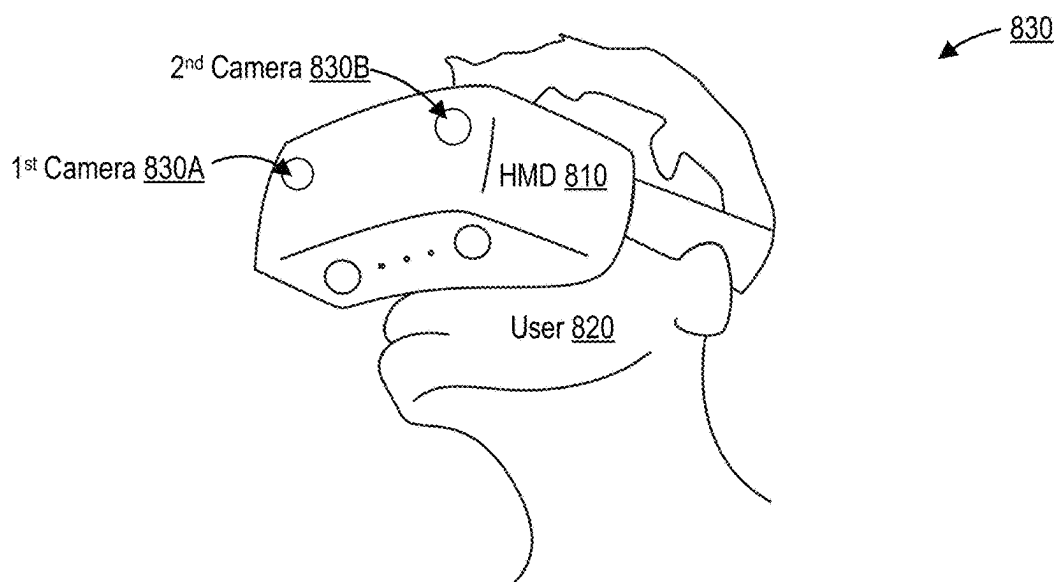
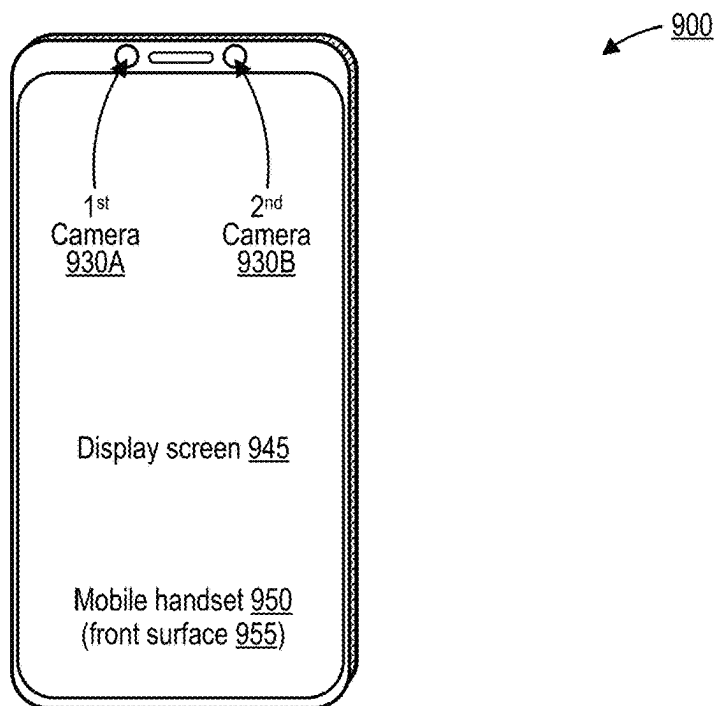
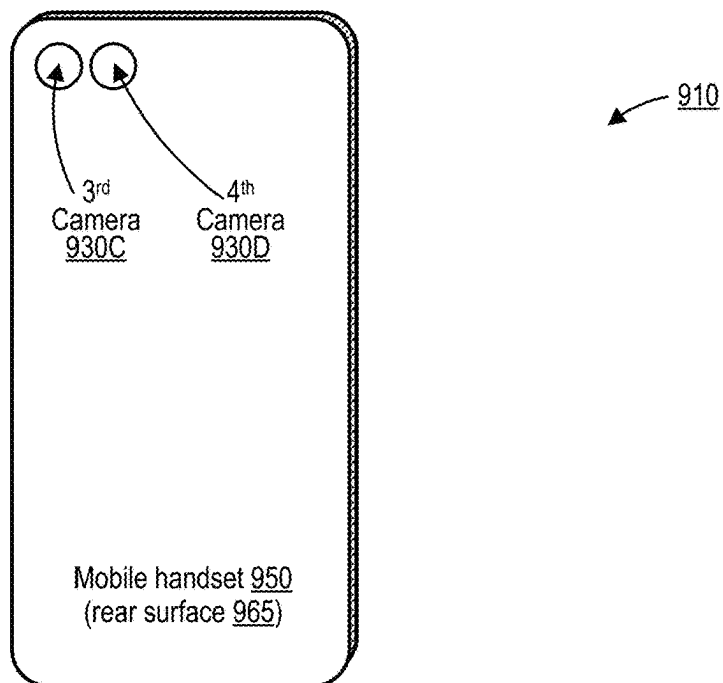


FIG. 8B

**FIG. 9A****FIG. 9B**

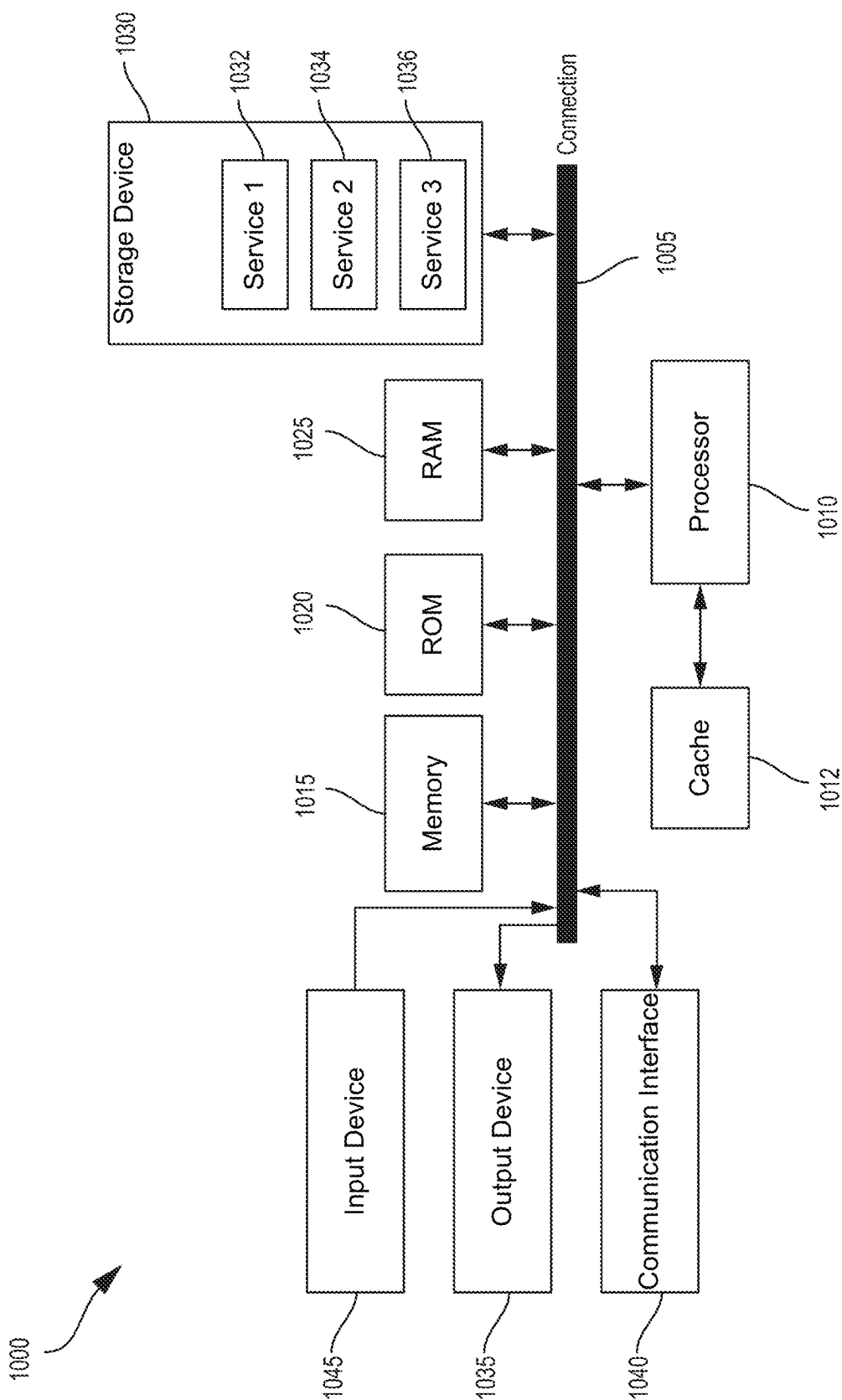


FIG. 10

1

EDGE-BASED GEOMETRY REPRESENTATION FOR CROSS LAYER APPLICATIONS

FIELD

This application is related to generating representations of objects. For example, aspects of the application relate to systems and techniques for an edge-based geometry representation for cross layer applications.

BACKGROUND

Extended reality (XR) technologies can be used to present virtual content to users, and/or can combine real environments from the physical world and virtual environments to provide users with XR experiences. The term XR can encompass virtual reality (VR), augmented reality (AR), mixed reality (MR), and the like. XR systems can allow users to experience XR environments by overlaying virtual content onto images of a real-world environment, which can be viewed by a user through an XR device (e.g., a head-mounted display (HMD), extended reality glasses, or other device). For example, an XR device can display an environment to a user. The environment is at least partially different from the real-world environment in which the user is in. The user can generally change their view of the environment interactively, for example by tilting or moving the XR device (e.g., the HMD or other device).

In some cases, an XR system can include a “see-through” display that allows the user to see their real-world environment based on light from the real-world environment passing through the display. In some cases, an XR system can include a “pass-through” display that allows the user to see their real-world environment, or a virtual environment based on their real-world environment, based on a view of the environment being captured by one or more cameras and displayed on the display. “See-through” or “pass-through” XR systems can be worn by users while the users are engaged in activities in their real-world environment.

In some cases, the user may be able to interact with virtual objects displayed within their view. These virtual objects may be presented in such a way that they appear to interact with the environment. For example, virtual text may be displayed such that the virtual text appears to be on a wall of the environment, or a virtual ball appears to bounce off of real objects in the environment. To allow such interactions, the XR system may track features in the environment. To help the XR system track features, the XR system may generate a representation of the environment based on, for example, sensor data from a variety of sensors, such as cameras, radar sensors, lidar sensors, and the like. Improvements for how a XR systems, and image processing systems generally, represent features and/or objects may thus be useful.

SUMMARY

Systems and techniques are described herein for protecting against malicious attacks in images. The following presents a simplified summary relating to one or more aspects disclosed herein. Thus, the following summary should not be considered an extensive overview relating to all contemplated aspects, nor should the following summary be considered to identify key or critical elements relating to all contemplated aspects or to delineate the scope associated with any particular aspect. Accordingly, the following sum-

2

mary presents certain concepts relating to one or more aspects relating to the mechanisms disclosed herein in a simplified form to precede the detailed description presented below.

5 Systems and techniques are described for an apparatus for image processing is provided. The apparatus includes a memory comprising instructions; and a processor coupled to the memory. The processor is configured to: obtain, from one or more image sensors, a first image of an environment; 10 determine semantic labels for a plurality of pixels of the first image based on whether each pixel of the plurality of pixels is associated with an object in the first image to generate a semantically segmented image; apply a two-dimensional line representation of the first image to the semantically segmented image to generate a labeled 2D line representation; back-project the labeled 2D line representation to three dimensions to generate a line representation model; fuse the line representation model and the semantically segmented 20 image to generate a labeled line representation model; and output the labeled line representation model.

In another example, a method for image generation is provided. The method includes: obtaining, from one or more image sensors, a first image of an environment; determining semantic labels for a plurality of pixels of the first image based on whether each pixel of the plurality of pixels is associated with an object in the first image to generate a semantically segmented image; applying a two-dimensional line representation of the first image to the semantically segmented image to generate a labeled 2D line representation; back-projecting the labeled 2D line representation to three dimensions to generate a line representation model; fusing the line representation model and the semantically segmented image to generate a labeled line representation 35 model; and outputting the labeled line representation model.

As another example, a non-transitory computer-readable medium is provided. The non-transitory computer-readable medium has stored thereon instructions that, when executed by at least one processor, cause the at least one processor to: 40 obtain, from one or more image sensors, a first image of an environment; determine semantic labels for a plurality of pixels of the first image based on whether each pixel of the plurality of pixels is associated with an object in the first image to generate a semantically segmented image; apply a two-dimensional line representation of the first image to the semantically segmented image to generate a labeled 2D line representation; back-project the labeled 2D line representation to three dimensions to generate a line representation model; fuse the line representation model and the semantically segmented image to generate a labeled line representation 45 model; and output the labeled line representation model.

In another example, an apparatus for image processing is provided. The apparatus includes: means for obtaining, from one or more image sensors, a first image of an environment; means for determining semantic labels for a plurality of pixels of the first image based on whether each pixel of the plurality of pixels is associated with an object in the first image to generate a semantically segmented image; means 50 for applying a two-dimensional line representation of the first image to the semantically segmented image to generate a labeled 2D line representation; means for back-projecting the labeled 2D line representation to three dimensions to generate a line representation model; means for fusing the line representation model and the semantically segmented image to generate a labeled line representation model; and means for outputting the labeled line representation model.

In some aspects, the apparatus can include or be part of an extended reality device (e.g., a virtual reality (VR) device, an augmented reality (AR) device, or a mixed reality (MR) device), a mobile device (e.g., a mobile telephone or other mobile device), a wearable device (e.g., a network-connected watch or other wearable device), a personal computer, a laptop computer, a server computer, a television, a video game console, or other device. In some aspects, the apparatus further includes at least one camera for capturing one or more images or video frames. For example, the apparatus can include a camera (e.g., an RGB camera) or multiple cameras for capturing one or more images and/or one or more videos including video frames. In some aspects, the apparatus includes a display for displaying one or more images, videos, notifications, or other displayable data. In some aspects, the apparatus includes a transmitter configured to transmit data or information over a transmission medium to at least one device. In some aspects, the processor includes a central processing unit (CPU), a graphics processing unit (GPU), a neural processing unit (NPU), or other processing device or component.

This summary is not intended to identify key or essential features of the claimed subject matter, nor is it intended to be used in isolation to determine the scope of the claimed subject matter. The subject matter should be understood by reference to appropriate portions of the entire specification of this patent, any or all drawings, and each claim.

The foregoing, together with other features and examples, will become more apparent upon referring to the following specification, claims, and accompanying drawings.

BRIEF DESCRIPTION OF THE DRAWINGS

Illustrative examples of the present application are described in detail below with reference to the following figures:

FIG. 1 is a block diagram illustrating an architecture of an image capture and processing system, in accordance with aspects of the present disclosure.

FIG. 2 is a diagram illustrating an architecture of an example extended reality (XR) system, in accordance with some aspects of the disclosure.

FIG. 3A-FIG. 4 are diagrams illustrating examples of neural networks, in accordance with some examples.

FIG. 5 illustrates an edge-based representation of objects in an environment, in accordance with aspects of the present disclosure.

FIG. 6 is a block diagram illustrating a process for generating an edge-based representation for cross-layer applications, in accordance with aspects of the present disclosure.

FIG. 7 is a flow diagram illustrating a process 700 for image processing, in accordance with aspects of the present disclosure.

FIG. 8A is a perspective diagram illustrating a head-mounted display (HMD) that performs feature tracking and/or visual simultaneous localization and mapping (VSLAM), in accordance with some examples.

FIG. 8B is a perspective diagram illustrating the head-mounted display (HMD) of FIG. 8A being worn by a user, in accordance with some examples.

FIG. 9A is a perspective diagram illustrating a front surface of a mobile device that performs feature tracking and/or visual simultaneous localization and mapping (VSLAM) using one or more front-facing cameras, in accordance with some examples.

FIG. 9B is a perspective diagram illustrating a rear surface of a mobile device 950, in accordance with aspects of the present disclosure.

FIG. 10 is a diagram illustrating an example of a system for implementing certain aspects of the present technology.

DETAILED DESCRIPTION

Certain aspects and examples of this disclosure are provided below. Some of these aspects and examples may be applied independently and some of them may be applied in combination as would be apparent to those of skill in the art. In the following description, for the purposes of explanation, specific details are set forth in order to provide a thorough understanding of subject matter of the application. However, it will be apparent that various examples may be practiced without these specific details. The figures and description are not intended to be restrictive.

The ensuing description provides illustrative examples only, and is not intended to limit the scope, applicability, or configuration of the disclosure. Rather, the ensuing description will provide those skilled in the art with an enabling description for implementing the illustrative examples. It should be understood that various changes may be made in the function and arrangement of elements without departing from the spirit and scope of the application as set forth in the appended claims.

Various types of systems can use a map of an environment. For example, an extended reality (XR) system can use the map of a real-world environment to accurately register virtual content to objects in the real-world environment. In another example, a vehicle (or computing systems of the vehicle) can use a map of a real-world environment for navigation purposes. Other types of systems can also generate and process maps of environments, including robotics systems, unmanned aerial vehicles, and/or other systems.

For example, XR systems or devices can provide virtual content to a user and/or can combine real-world or physical environments and virtual environments (made up of virtual content) to provide users with XR experiences. The real-world environment can include real-world objects (also referred to as physical objects), such as people, vehicles, buildings, tables, chairs, and/or other real-world or physical objects. XR systems or devices can facilitate interaction with different types of XR environments (e.g., a user can use an XR system or device to interact with an XR environment). XR systems can include virtual reality (VR) systems facilitating interactions with VR environments, augmented reality (AR) systems facilitating interactions with AR environments, mixed reality (MR) systems facilitating interactions with MR environments, and/or other XR systems. Examples of XR systems or devices include head-mounted displays (HMDs), smart glasses, among others. In some cases, an XR system can track parts of the user (e.g., a hand and/or fingertips of a user) to allow the user to interact with items of virtual content.

AR is a technology that provides virtual or computer-generated content (referred to as AR content) over the user's view of a physical, real-world scene or environment. AR content can include virtual content, such as video, images, graphic content, location data (e.g., global positioning system (GPS) data or other location data), sounds, any combination thereof, and/or other augmented content. An AR system or device is designed to enhance (or augment), rather than to replace, a person's current perception of reality. For example, a user can see a real stationary or moving physical object through an AR device display, but the user's visual

perception of the physical object may be augmented or enhanced by a virtual image of that object (e.g., a real-world car replaced by a virtual image of a DeLorean), by AR content added to the physical object (e.g., virtual wings added to a live animal), by AR content displayed relative to the physical object (e.g., informational virtual content displayed near a sign on a building, a virtual coffee cup virtually anchored to (e.g., placed on top of) a real-world table in one or more images, etc.), and/or by displaying other types of AR content. Various types of AR systems can be used for gaming, entertainment, and/or other applications.

In some cases, a XR system generate a map of an environment based on feature points. In some cases, the map may be stored as a mesh representation of the feature points. This mesh representation can be fairly memory and/or processor intensive to handle. For example, an XR system may need to store information about numerous feature points, and how the feature points relate to each other, as a mesh made of multiple points. The XR system can process the mesh, for example, to generate a view for display based on the mesh. As noted above, other types of systems that can generate and process maps of environments can include robotics systems, vehicles (or computing systems of vehicles), unmanned aerial vehicles, and/or other systems.

Systems, apparatuses, electronic devices, methods (also referred to as processes), and computer-readable media (collectively referred to herein as “systems and techniques”) are described herein for generating an edge-based geometry representation of objects in an environment. The systems can include an XR system, a vehicle (or a computing system of the vehicle), an unmanned aerial vehicle, and/or other type of system. In some aspects, it may be useful to represent certain objects using an edge-based representation (e.g., wireframe) where a structure of objects in the environment may be represented by three-dimensional lines rather than many points. For example, the systems and techniques can generate an edge-based representation of one or more objects in an image. The edge-based representation and in some cases a semantic segmentation map of the image can be processed (e.g., using a machine learning system, such as a neural network algorithm or model) to generate or update an environment map. The semantic segmentation map can include labels (e.g., a respective label for each pixel of the image) for only objects of interest in the image (e.g., for objects depicted in the edge-based representation).

In some cases, using an edge-based geometry representation may allow for a more compact and space efficient way to store a map of the environment around a device. In some examples, by using three-dimensional lines representing a structure of objects, views based on the object may be rendered directly from the map.

Various aspects of the application will be described with respect to the figures. FIG. 1 is a block diagram illustrating an architecture of an image capture and processing system 100. The image capture and processing system 100 includes various components that are used to capture and process images of scenes (e.g., an image of a scene 110). The image capture and processing system 100 can capture standalone images (or photographs) and/or can capture videos that include multiple images (or video frames) in a particular sequence. In some cases, the lens 115 and image sensor 130 can be associated with an optical axis. In one illustrative example, the photosensitive area of the image sensor 130 (e.g., the photodiodes) and the lens 115 can both be centered on the optical axis. A lens 115 of the image capture and processing system 100 faces a scene 110 and receives light from the scene 110. The lens 115 bends incoming light from

the scene toward the image sensor 130. The light received by the lens 115 passes through an aperture. In some cases, the aperture (e.g., the aperture size) is controlled by one or more control mechanisms 120 and is received by an image sensor 130. In some cases, the aperture can have a fixed size.

The one or more control mechanisms 120 may control exposure, focus, and/or zoom based on information from the image sensor 130 and/or based on information from the image processor 150. The one or more control mechanisms 120 may include multiple mechanisms and components; for instance, the control mechanisms 120 may include one or more exposure control mechanisms 125A, one or more focus control mechanisms 125B, and/or one or more zoom control mechanisms 125C. The one or more control mechanisms 120 may also include additional control mechanisms besides those that are illustrated, such as control mechanisms controlling analog gain, flash, HDR, depth of field, and/or other image capture properties.

The focus control mechanism 125B of the control mechanisms 120 can obtain a focus setting. In some examples, focus control mechanism 125B store the focus setting in a memory register. Based on the focus setting, the focus control mechanism 125B can adjust the position of the lens 115 relative to the position of the image sensor 130. For example, based on the focus setting, the focus control mechanism 125B can move the lens 115 closer to the image sensor 130 or farther from the image sensor 130 by actuating a motor or servo (or other lens mechanism), thereby adjusting focus. In some cases, additional lenses may be included in the image capture and processing system 100, such as one or more microlenses over each photodiode of the image sensor 130, which each bend the light received from the lens 115 toward the corresponding photodiode before the light reaches the photodiode. The focus setting may be determined via contrast detection autofocus (CDAF), phase detection autofocus (PDAF), hybrid autofocus (HAF), or some combination thereof. The focus setting may be determined using the control mechanism 120, the image sensor 130, and/or the image processor 150. The focus setting may be referred to as an image capture setting and/or an image processing setting. In some cases, the lens 115 can be fixed relative to the image sensor and focus control mechanism 125B can be omitted without departing from the scope of the present disclosure.

The exposure control mechanism 125A of the control mechanisms 120 can obtain an exposure setting. In some cases, the exposure control mechanism 125A stores the exposure setting in a memory register. Based on this exposure setting, the exposure control mechanism 125A can control a size of the aperture (e.g., aperture size or f/stop), a duration of time for which the aperture is open (e.g., exposure time or shutter speed), a duration of time for which the sensor collects light (e.g., exposure time or electronic shutter speed), a sensitivity of the image sensor 130 (e.g., ISO speed or film speed), analog gain applied by the image sensor 130, or any combination thereof. The exposure setting may be referred to as an image capture setting and/or an image processing setting.

The zoom control mechanism 125C of the control mechanisms 120 can obtain a zoom setting. In some examples, the zoom control mechanism 125C stores the zoom setting in a memory register. Based on the zoom setting, the zoom control mechanism 125C can control a focal length of an assembly of lens elements (lens assembly) that includes the lens 115 and one or more additional lenses. For example, the zoom control mechanism 125C can control the focal length of the lens assembly by actuating one or more motors or

servos (or other lens mechanism) to move one or more of the lenses relative to one another. The zoom setting may be referred to as an image capture setting and/or an image processing setting. In some examples, the lens assembly may include a parfocal zoom lens or a varifocal zoom lens. In some examples, the lens assembly may include a focusing lens (which can be lens **115** in some cases) that receives the light from the scene **110** first, with the light then passing through an afocal zoom system between the focusing lens (e.g., lens **115**) and the image sensor **130** before the light reaches the image sensor **130**. The afocal zoom system may, in some cases, include two positive (e.g., converging, convex) lenses of equal or similar focal length (e.g., within a threshold difference of one another) with a negative (e.g., diverging, concave) lens between them. In some cases, the zoom control mechanism **125C** moves one or more of the lenses in the afocal zoom system, such as the negative lens and one or both of the positive lenses. In some cases, zoom control mechanism **125C** can control the zoom by capturing an image from an image sensor of a plurality of image sensors (e.g., including image sensor **130**) with a zoom corresponding to the zoom setting. For example, image processing system **100** can include a wide angle image sensor with a relatively low zoom and a telephoto image sensor with a greater zoom. In some cases, based on the selected zoom setting, the zoom control mechanism **125C** can capture images from a corresponding sensor.

The image sensor **130** includes one or more arrays of photodiodes or other photosensitive elements. Each photodiode measures an amount of light that eventually corresponds to a particular pixel in the image produced by the image sensor **130**. In some cases, different photodiodes may be covered by different filters. In some cases, different photodiodes can be covered in color filters, and may thus measure light matching the color of the filter covering the photodiode. Various color filter arrays can be used, including a Bayer color filter array, a quad color filter array (also referred to as a quad Bayer color filter array or QCFA), and/or any other color filter array. For instance, Bayer color filters include red color filters, blue color filters, and green color filters, with each pixel of the image generated based on red light data from at least one photodiode covered in a red color filter, blue light data from at least one photodiode covered in a blue color filter, and green light data from at least one photodiode covered in a green color filter.

Returning to FIG. 1, other types of color filters may use yellow, magenta, and/or cyan (also referred to as “emerald”) color filters instead of or in addition to red, blue, and/or green color filters. In some cases, some photodiodes may be configured to measure infrared (IR) light. In some implementations, photodiodes measuring IR light may not be covered by any filter, thus allowing IR photodiodes to measure both visible (e.g., color) and IR light. In some examples, IR photodiodes may be covered by an IR filter, allowing IR light to pass through and blocking light from other parts of the frequency spectrum (e.g., visible light, color). Some image sensors (e.g., image sensor **130**) may lack filters (e.g., color, IR, or any other part of the light spectrum) altogether and may instead use different photodiodes throughout the pixel array (in some cases vertically stacked). The different photodiodes throughout the pixel array can have different spectral sensitivity curves, therefore responding to different wavelengths of light. Monochrome image sensors may also lack filters and therefore lack color depth.

In some cases, the image sensor **130** may alternately or additionally include opaque and/or reflective masks that

block light from reaching certain photodiodes, or portions of certain photodiodes, at certain times and/or from certain angles. In some cases, opaque and/or reflective masks may be used for phase detection autofocus (PDAF). In some cases, the opaque and/or reflective masks may be used to block portions of the electromagnetic spectrum from reaching the photodiodes of the image sensor (e.g., an IR cut filter, a UV cut filter, a band-pass filter, low-pass filter, high-pass filter, or the like). The image sensor **130** may also include an analog gain amplifier to amplify the analog signals output by the photodiodes and/or an analog to digital converter (ADC) to convert the analog signals output of the photodiodes (and/or amplified by the analog gain amplifier) into digital signals. In some cases, certain components or functions discussed with respect to one or more of the control mechanisms **120** may be included instead or additionally in the image sensor **130**. The image sensor **130** may be a charge-coupled device (CCD) sensor, an electron-multiplying CCD (EMCCD) sensor, an active-pixel sensor (APS), a complimentary metal-oxide semiconductor (CMOS), an N-type metal-oxide semiconductor (NMOS), a hybrid CCD/CMOS sensor (e.g., sCMOS), or some other combination thereof.

The image processor **150** may include one or more processors, such as one or more image signal processors (ISPs) (including ISP **154**), one or more host processors (including host processor **152**), and/or one or more of any other type of processor **1010** discussed with respect to the computing system **1000** of FIG. 10. The host processor **152** can be a digital signal processor (DSP) and/or other type of processor. In some implementations, the image processor **150** is a single integrated circuit or chip (e.g., referred to as a system-on-chip or SoC) that includes the host processor **152** and the ISP **154**. In some cases, the chip can also include one or more input/output ports (e.g., input/output (I/O) ports **156**), central processing units (CPUs), graphics processing units (GPUs), broadband modems (e.g., 3G, 4G or LTE, 5G, etc.), memory, connectivity components (e.g., Bluetooth™, Global Positioning System (GPS), etc.), any combination thereof, and/or other components. The I/O ports **156** can include any suitable input/output ports or interface according to one or more protocol or specification, such as an Inter-Integrated Circuit 2 (I2C) interface, an Inter-Integrated Circuit 3 (I3C) interface, a Serial Peripheral Interface (SPI) interface, a serial General Purpose Input/Output (GPIO) interface, a Mobile Industry Processor Interface (MIPI) (such as a MIPI CSI-2 physical (PHY) layer port or interface, an Advanced High-performance Bus (AHB) bus, any combination thereof, and/or other input/output port. In one illustrative example, the host processor **152** can communicate with the image sensor **130** using an I2C port, and the ISP **154** can communicate with the image sensor **130** using an MIPI port.

The image processor **150** may perform a number of tasks, such as de-mosaicing, color space conversion, image frame downsampling, pixel interpolation, automatic exposure (AE) control, automatic gain control (AGC), CDAF, PDAF, automatic white balance, merging of image frames to form an HDR image, image recognition, object recognition, feature recognition, receipt of inputs, managing outputs, managing memory, or some combination thereof. The image processor **150** may store image frames and/or processed images in random access memory (RAM) **140/1025**, read-only memory (ROM) **145/1020**, a cache, a memory unit, another storage device, or some combination thereof.

Various input/output (I/O) devices **160** may be connected to the image processor **150**. The I/O devices **160** can include a display screen, a keyboard, a keypad, a touchscreen, a

trackpad, a touch-sensitive surface, a printer, any other output devices, any other input devices, or some combination thereof. In some cases, a caption may be input into the image processing device **105B** through a physical keyboard or keypad of the I/O devices **160**, or through a virtual keyboard or keypad of a touchscreen of the I/O devices **160**. The I/O devices **160** may include one or more ports, jacks, or other connectors that enable a wired connection between the image capture and processing system **100** and one or more peripheral devices, over which the image capture and processing system **100** may receive data from the one or more peripheral device and/or transmit data to the one or more peripheral devices. The peripheral devices may include any of the previously-discussed types of I/O devices **160** and may themselves be considered I/O devices **160** once they are coupled to the ports, jacks, wireless transceivers, or other wired and/or wireless connectors.

In some cases, the image capture and processing system **100** may be a single device. In some cases, the image capture and processing system **100** may be two or more separate devices, including an image capture device **105A** (e.g., a camera) and an image processing device **105B** (e.g., a computing device coupled to the camera). In some implementations, the image capture device **105A** and the image processing device **105B** may be coupled together, for example via one or more wires, cables, or other electrical connectors, and/or wirelessly via one or more wireless transceivers. In some implementations, the image capture device **105A** and the image processing device **105B** may be disconnected from one another.

As shown in FIG. 1, a vertical dashed line divides the image capture and processing system **100** of FIG. 1 into two portions that represent the image capture device **105A** and the image processing device **105B**, respectively. The image capture device **105A** includes the lens **115**, control mechanisms **120**, and the image sensor **130**. The image processing device **105B** includes the image processor **150** (including the ISP **154** and the host processor **152**), the RAM **140**, the ROM **145**, and the I/O devices **160**. In some cases, certain components illustrated in the image capture device **105A**, such as the ISP **154** and/or the host processor **152**, may be included in the image capture device **105A**.

The image capture and processing system **100** can include an electronic device, such as a mobile or stationary telephone handset (e.g., smartphone, cellular telephone, or the like), a desktop computer, a laptop or notebook computer, a tablet computer, a set-top box, a television, a camera, a display device, a digital media player, a video gaming console, a video streaming device, an Internet Protocol (IP) camera, or any other suitable electronic device. In some examples, the image capture and processing system **100** can include one or more wireless transceivers for wireless communications, such as cellular network communications, 802.10 wi-fi communications, wireless local area network (WLAN) communications, or some combination thereof. In some implementations, the image capture device **105A** and the image processing device **105B** can be different devices. For instance, the image capture device **105A** can include a camera device and the image processing device **105B** can

include a computing device, such as a mobile handset, a desktop computer, or other computing device.

While the image capture and processing system **100** is shown to include certain components, one of ordinary skill will appreciate that the image capture and processing system **100** can include more components than those shown in FIG. 1. The components of the image capture and processing system **100** can include software, hardware, or one or more combinations of software and hardware. For example, in some implementations, the components of the image capture and processing system **100** can include and/or can be implemented using electronic circuits or other electronic hardware, which can include one or more programmable electronic circuits (e.g., microprocessors, GPUs, DSPs, CPUs, and/or other suitable electronic circuits), and/or can include and/or be implemented using computer software, firmware, or any combination thereof, to perform the various operations described herein. The software and/or firmware can include one or more instructions stored on a computer-readable storage medium and executable by one or more processors of the electronic device implementing the image capture and processing system **100**.

In some examples, the extended reality (XR) system **200** of FIG. 2 can include the image capture and processing system **100**, the image capture device **105A**, the image processing device **105B**, or a combination thereof.

FIG. 2 is a diagram illustrating an architecture of an example extended reality (XR) system **200**, in accordance with some aspects of the disclosure. The XR system **200** can run (or execute) XR applications and implement XR operations. In some examples, the XR system **200** can perform tracking and localization, mapping of an environment in the physical world (e.g., a scene), and/or positioning and rendering of virtual content on a display **209** (e.g., a screen, visible plane/region, and/or other display) as part of an XR experience. For example, the XR system **200** can generate a map (e.g., a three-dimensional (3D) map) of an environment in the physical world, track a pose (e.g., location and position) of the XR system **200** relative to the environment (e.g., relative to the 3D map of the environment), position and/or anchor virtual content in a specific location(s) on the map of the environment, and render the virtual content on the display **209** such that the virtual content appears to be at a location in the environment corresponding to the specific location on the map of the scene where the virtual content is positioned and/or anchored. The display **209** can include a glass, a screen, a lens, a projector, and/or other display mechanism that allows a user to see the real-world environment and also allows XR content to be overlaid, overlapped, blended with, or otherwise displayed thereon.

In this illustrative example, the XR system **200** includes one or more image sensors **202**, an accelerometer **204**, a gyroscope **206**, storage **207**, compute components **210**, an XR engine **220**, an image processing engine **224**, a rendering engine **226**, and a communications engine **228**. It should be noted that the components **202-228** shown in FIG. 2 are non-limiting examples provided for illustrative and explanation purposes, and other examples can include more, fewer, or different components than those shown in FIG. 2. For example, in some cases, the XR system **200** can include one or more other sensors (e.g., one or more inertial measurement units (IMUs), radars, light detection and ranging (LIDAR) sensors, radio detection and ranging (RADAR) sensors, sound detection and ranging (SODAR) sensors, sound navigation and ranging (SONAR) sensors, audio sensors, etc.), one or more display devices, one or more other processing engines, one or more other hardware compo-

11

nents, and/or one or more other software and/or hardware components that are not shown in FIG. 2. While various components of the XR system 200, such as the image sensor 202, may be referenced in the singular form herein, it should be understood that the XR system 200 may include multiple of any component discussed herein (e.g., multiple image sensors 202).

The XR system 200 includes or is in communication with (wired or wirelessly) an input device 208. The input device 208 can include any suitable input device, such as a touchscreen, a pen or other pointer device, a keyboard, a mouse a button or key, a microphone for receiving voice commands, a gesture input device for receiving gesture commands, a video game controller, a steering wheel, a joystick, a set of buttons, a trackball, a remote control, any other input device 1045 discussed herein, or any combination thereof. In some cases, the image sensor 202 can capture images that can be processed for interpreting gesture commands.

The XR system 200 can also communicate with one or more other electronic devices (wired or wirelessly). For example, communications engine 228 can be configured to manage connections and communicate with one or more electronic devices. In some cases, the communications engine 228 can correspond to the communications interface 1040 of FIG. 10.

In some implementations, the one or more image sensors 202, the accelerometer 204, the gyroscope 206, storage 207, compute components 210, XR engine 220, image processing engine 224, and rendering engine 226 can be part of the same computing device. For example, in some cases, the one or more image sensors 202, the accelerometer 204, the gyroscope 206, storage 207, compute components 210, XR engine 220, image processing engine 224, and rendering engine 226 can be integrated into an HMD, extended reality glasses, smartphone, laptop, tablet computer, gaming system, and/or any other computing device. However, in some implementations, the one or more image sensors 202, the accelerometer 204, the gyroscope 206, storage 207, compute components 210, XR engine 220, image processing engine 224, and rendering engine 226 can be part of two or more separate computing devices. For example, in some cases, some of the components 202-226 can be part of, or implemented by, one computing device and the remaining components can be part of, or implemented by, one or more other computing devices.

The storage 207 can be any storage device(s) for storing data. Moreover, the storage 207 can store data from any of the components of the XR system 200. For example, the storage 207 can store data from the image sensor 202 (e.g., image or video data), data from the accelerometer 204 (e.g., measurements), data from the gyroscope 206 (e.g., measurements), data from the compute components 210 (e.g., processing parameters, preferences, virtual content, rendering content, scene maps, tracking and localization data, object detection data, privacy data, XR application data, face recognition data, occlusion data, etc.), data from the XR engine 220, data from the image processing engine 224, and/or data from the rendering engine 226 (e.g., output frames). In some examples, the storage 207 can include a buffer for storing frames for processing by the compute components 210.

The one or more compute components 210 can include a central processing unit (CPU) 212, a graphics processing unit (GPU) 214, a digital signal processor (DSP) 216, an image signal processor (ISP) 218, and/or other processor (e.g., a neural processing unit (NPU) implementing one or more trained neural networks). The compute components

12

210 can perform various operations such as image enhancement, computer vision, graphics rendering, extended reality operations (e.g., tracking, localization, pose estimation, mapping, content anchoring, content rendering, etc.), image and/or video processing, sensor processing, recognition (e.g., text recognition, facial recognition, object recognition, feature recognition, tracking or pattern recognition, scene recognition, occlusion detection, etc.), trained machine learning operations, filtering, and/or any of the various operations described herein. In some examples, the compute components 210 can implement (e.g., control, operate, etc.) the XR engine 220, the image processing engine 224, and the rendering engine 226. In other examples, the compute components 210 can also implement one or more other processing engines.

The image sensor 202 can include any image and/or video sensors or capturing devices. In some examples, the image sensor 202 can be part of a multiple-camera assembly, such as a dual-camera assembly. The image sensor 202 can capture image and/or video content (e.g., raw image and/or video data), which can then be processed by the compute components 210, the XR engine 220, the image processing engine 224, and/or the rendering engine 226 as described herein. In some examples, the image sensors 202 may include an image capture and processing system 100, an image capture device 105A, an image processing device 105B, or a combination thereof.

In some examples, the image sensor 202 can capture image data and can generate images (also referred to as frames) based on the image data and/or can provide the image data or frames to the XR engine 220, the image processing engine 224, and/or the rendering engine 226 for processing. An image or frame can include a video frame of a video sequence or a still image. An image or frame can include a pixel array representing a scene. For example, an image can be a red-green-blue (RGB) image having red, green, and blue color components per pixel; a luma, chroma-red, chroma-blue (YCbCr) image having a luma component and two chroma (color) components (chroma-red and chroma-blue) per pixel; or any other suitable type of color or monochrome image.

In some cases, the image sensor 202 (and/or other camera of the XR system 200) can be configured to also capture depth information. For example, in some implementations, the image sensor 202 (and/or other camera) can include an RGB-depth (RGB-D) camera. In some cases, the XR system 200 can include one or more depth sensors (not shown) that are separate from the image sensor 202 (and/or other camera) and that can capture depth information. For instance, such a depth sensor can obtain depth information independently from the image sensor 202. In some examples, a depth sensor can be physically installed in the same general location as the image sensor 202, but may operate at a different frequency or frame rate from the image sensor 202. In some examples, a depth sensor can take the form of a light source that can project a structured or textured light pattern, which may include one or more narrow bands of light, onto one or more objects in a scene. Depth information can then be obtained by exploiting geometrical distortions of the projected pattern caused by the surface shape of the object. In one example, depth information may be obtained from stereo sensors such as a combination of an infra-red structured light projector and an infra-red camera registered to a camera (e.g., an RGB camera).

The XR system 200 can also include other sensors in its one or more sensors. The one or more sensors can include one or more accelerometers (e.g., accelerometer 204), one or

more gyroscopes (e.g., gyroscope 206), and/or other sensors. The one or more sensors can provide velocity, orientation, and/or other position-related information to the compute components 210. For example, the accelerometer 204 can detect acceleration by the XR system 200 and can generate acceleration measurements based on the detected acceleration. In some cases, the accelerometer 204 can provide one or more translational vectors (e.g., up/down, left/right, forward/back) that can be used for determining a position or pose of the XR system 200. The gyroscope 206 can detect and measure the orientation and angular velocity of the XR system 200. For example, the gyroscope 206 can be used to measure the pitch, roll, and yaw of the XR system 200. In some cases, the gyroscope 206 can provide one or more rotational vectors (e.g., pitch, yaw, roll). In some examples, the image sensor 202 and/or the XR engine 220 can use measurements obtained by the accelerometer 204 (e.g., one or more translational vectors) and/or the gyroscope 206 (e.g., one or more rotational vectors) to calculate the pose of the XR system 200. As previously noted, in other examples, the XR system 200 can also include other sensors, such as an inertial measurement unit (IMU), a magnetometer, a gaze and/or eye tracking sensor, a machine vision sensor, a smart scene sensor, a speech recognition sensor, an impact sensor, a shock sensor, a position sensor, a tilt sensor, etc.

As noted above, in some cases, the one or more sensors can include at least one IMU. An IMU is an electronic device that measures the specific force, angular rate, and/or the orientation of the XR system 200, using a combination of one or more accelerometers, one or more gyroscopes, and/or one or more magnetometers. In some examples, the one or more sensors can output measured information associated with the capture of an image captured by the image sensor 202 (and/or other camera of the XR system 200) and/or depth information obtained using one or more depth sensors of the XR system 200.

The output of one or more sensors (e.g., the accelerometer 204, the gyroscope 206, one or more IMUs, and/or other sensors) can be used by the XR engine 220 to determine a pose of the XR system 200 (also referred to as the head pose) and/or the pose of the image sensor 202 (or other camera of the XR system 200). In some cases, the pose of the XR system 200 and the pose of the image sensor 202 (or other camera) can be the same. The pose of image sensor 202 refers to the position and orientation of the image sensor 202 relative to a frame of reference (e.g., with respect to the scene 110). In some implementations, the camera pose can be determined for 6-Degrees Of Freedom (6DoF), which refers to three translational components (e.g., which can be given by X (horizontal), Y (vertical), and Z (depth) coordinates relative to a frame of reference, such as the image plane) and three angular components (e.g., roll, pitch, and yaw relative to the same frame of reference). In some implementations, the camera pose can be determined for 3-Degrees Of Freedom (3DoF), which refers to the three angular components (e.g., roll, pitch, and yaw).

In some cases, a device tracker (not shown) can use the measurements from the one or more sensors and image data from the image sensor 202 to track a pose (e.g., a 6DoF pose) of the XR system 200. For example, the device tracker can fuse visual data (e.g., using a visual tracking solution) from the image data with inertial data from the measurements to determine a position and motion of the XR system 200 relative to the physical world (e.g., the scene) and a map of the physical world. As described below, in some examples, when tracking the pose of the XR system 200, the device tracker can generate a three-dimensional (3D) map of

the scene (e.g., the real world) and/or generate updates for a 3D map of the scene. The 3D map updates can include, for example and without limitation, new or updated features and/or feature or landmark points associated with the scene and/or the 3D map of the scene, localization updates identifying or updating a position of the XR system 200 within the scene and the 3D map of the scene, etc. The 3D map can provide a digital representation of a scene in the real/physical world. In some examples, the 3D map can anchor location-based objects and/or content to real-world coordinates and/or objects. The XR system 200 can use a mapped scene (e.g., a scene in the physical world represented by, and/or associated with, a 3D map) to merge the physical and virtual worlds and/or merge virtual content or objects with the physical environment.

In some aspects, the pose of image sensor 202 and/or the XR system 200 as a whole can be determined and/or tracked by the compute components 210 using a visual tracking solution based on images captured by the image sensor 202 (and/or other camera of the XR system 200). For instance, in some examples, the compute components 210 can perform tracking using computer vision-based tracking, model-based tracking, and/or simultaneous localization and mapping (SLAM) techniques. For instance, the compute components 210 can perform SLAM or can be in communication (wired or wireless) with a SLAM system (not shown). SLAM refers to a class of techniques where a map of an environment (e.g., a map of an environment being modeled by XR system 200) is created while simultaneously tracking the pose of a camera (e.g., image sensor 202) and/or the XR system 200 relative to that map. The map can be referred to as a SLAM map, and can be three-dimensional (3D). The SLAM techniques can be performed using color or grayscale image data captured by the image sensor 202 (and/or other camera of the XR system 200), and can be used to generate estimates of 6DoF pose measurements of the image sensor 202 and/or the XR system 200. Such a SLAM technique configured to perform 6DoF tracking can be referred to as 6DoF SLAM. In some cases, the output of the one or more sensors (e.g., the accelerometer 204, the gyroscope 206, one or more IMUs, and/or other sensors) can be used to estimate, correct, and/or otherwise adjust the estimated pose.

In some cases, the 6DoF SLAM (e.g., 6DoF tracking) can associate features observed from certain input images from the image sensor 202 (and/or other camera) to the SLAM map. For example, 6DoF SLAM can use feature point associations from an input image to determine the pose (position and orientation) of the image sensor 202 and/or XR system 200 for the input image. 6DoF mapping can also be performed to update the SLAM map. In some cases, the SLAM map maintained using the 6DoF SLAM can contain 3D feature points triangulated from two or more images. For example, key frames can be selected from input images or a video stream to represent an observed scene. For every key frame, a respective 6DoF camera pose associated with the image can be determined. The pose of the image sensor 202 and/or the XR system 200 can be determined by projecting features from the 3D SLAM map into an image or video frame and updating the camera pose from verified 2D-3D correspondences.

In one illustrative example, the compute components 210 can extract feature points from certain input images (e.g., every input image, a subset of the input images, etc.) or from each key frame. A feature point (also referred to as a registration point) as used herein is a distinctive or identifiable part of an image, such as a part of a hand, an edge of

15

a table, among others. Features extracted from a captured image can represent distinct feature points along three-dimensional space (e.g., coordinates on X, Y, and Z-axes), and every feature point can have an associated feature location. The feature points in key frames either match (are the same or correspond to) or fail to match the feature points of previously-captured input images or key frames. Feature detection can be used to detect the feature points. Feature detection can include an image processing operation used to examine one or more pixels of an image to determine whether a feature exists at a particular pixel. Feature detection can be used to process an entire captured image or certain portions of an image. For each image or key frame, once features have been detected, a local image patch around the feature can be extracted. Features may be extracted using any suitable technique, such as Scale Invariant Feature Transform (SIFT) (which localizes features and generates their descriptions), Learned Invariant Feature Transform (LIFT), Speed Up Robust Features (SURF), Gradient Location-Orientation histogram (GLOH), Oriented Fast and Rotated Brief (ORB), Binary Robust Invariant Scalable Keypoints (BRISK), Fast Retina Keypoint (FREAK), KAZE, Accelerated KAZE (AKAZE), Normalized Cross Correlation (NCC), descriptor matching, another suitable technique, or a combination thereof.

As one illustrative example, the compute components 210 can extract feature points corresponding to a mobile device (e.g., mobile device 440 of FIG. 4, mobile device 540 of FIG. 5), or the like. In some cases, feature points corresponding to the mobile device can be tracked to determine a pose of the mobile device. As described in more detail below, the pose of the mobile device can be used to determine a location for projection of AR media content that can enhance media content displayed on a display of the mobile device.

In some cases, the XR system 200 can also track the hand and/or fingers of the user to allow the user to interact with and/or control virtual content in a virtual environment. For example, the XR system 200 can track a pose and/or movement of the hand and/or fingertips of the user to identify or translate user interactions with the virtual environment. The user interactions can include, for example and without limitation, moving an item of virtual content, resizing the item of virtual content, selecting an input interface element in a virtual user interface (e.g., a virtual representation of a mobile phone, a virtual keyboard, and/or other virtual interface), providing an input through a virtual user interface, etc.

A neural network is an example of a machine learning system, and a neural network can include an input layer, one or more hidden layers, and an output layer. Data is provided from input nodes of the input layer, processing is performed by hidden nodes of the one or more hidden layers, and an output is produced through output nodes of the output layer. Deep learning networks typically include multiple hidden layers. Each layer of the neural network can include feature maps or activation maps that can include artificial neurons (or nodes). A feature map can include a filter, a kernel, or the like. The nodes can include one or more weights used to indicate an importance of the nodes of one or more of the layers. In some cases, a deep learning network can have a series of many hidden layers, with early layers being used to determine simple and low level characteristics of an input, and later layers building up a hierarchy of more complex and abstract characteristics.

A deep learning architecture may learn a hierarchy of features. If presented with visual data, for example, the first

16

layer may learn to recognize relatively simple features, such as edges, in the input stream. In another example, if presented with auditory data, the first layer may learn to recognize spectral power in specific frequencies. The second layer, taking the output of the first layer as input, may learn to recognize combinations of features, such as simple shapes for visual data or combinations of sounds for auditory data. For instance, higher layers may learn to represent complex shapes in visual data or words in auditory data. Still higher layers may learn to recognize common visual objects or spoken phrases.

Deep learning architectures may perform especially well when applied to problems that have a natural hierarchical structure. For example, the classification of motorized vehicles may benefit from first learning to recognize wheels, windshields, and other features. These features may be combined at higher layers in different ways to recognize cars, trucks, and airplanes.

Neural networks may be designed with a variety of connectivity patterns. In feed-forward networks, information is passed from lower to higher layers, with each neuron in a given layer communicating to neurons in higher layers. A hierarchical representation may be built up in successive layers of a feed-forward network, as described above. Neural networks may also have recurrent or feedback (also called top-down) connections. In a recurrent connection, the output from a neuron in a given layer may be communicated to another neuron in the same layer. A recurrent architecture may be helpful in recognizing patterns that span more than one of the input data chunks that are delivered to the neural network in a sequence. A connection from a neuron in a given layer to a neuron in a lower layer is called a feedback (or top-down) connection. A network with many feedback connections may be helpful when the recognition of a high-level concept may aid in discriminating the particular low-level features of an input. The connections between layers of a neural network may be fully connected or locally connected. Various examples of neural network architectures are described below with respect to FIG. 3A-FIG. 4.

Neural networks may be designed with a variety of connectivity patterns. In feed-forward networks, information is passed from lower to higher layers, with each neuron in a given layer communicating to neurons in higher layers. A hierarchical representation may be built up in successive layers of a feed-forward network, as described above. Neural networks may also have recurrent or feedback (also called top-down) connections. In a recurrent connection, the output from a neuron in a given layer may be communicated to another neuron in the same layer. A recurrent architecture may be helpful in recognizing patterns that span more than one of the input data chunks that are delivered to the neural network in a sequence. A connection from a neuron in a given layer to a neuron in a lower layer is called a feedback (or top-down) connection. A network with many feedback connections may be helpful when the recognition of a high-level concept may aid in discriminating the particular low-level features of an input.

The connections between layers of a neural network may be fully connected or locally connected. FIG. 3A illustrates an example of a fully connected neural network 302. In a fully connected neural network 302, a neuron in a first layer may communicate its output to every neuron in a second layer, so that each neuron in the second layer will receive input from every neuron in the first layer. FIG. 3B illustrates an example of a locally connected neural network 304. In a locally connected neural network 304, a neuron in a first layer may be connected to a limited number of neurons in

17

the second layer. More generally, a locally connected layer of the locally connected neural network **304** may be configured so that each neuron in a layer will have the same or a similar connectivity pattern, but with connections strengths that may have different values (e.g., **310**, **312**, **314**, and **316**). The locally connected connectivity pattern may give rise to spatially distinct receptive fields in a higher layer, because the higher layer neurons in a given region may receive inputs that are tuned through training to the properties of a restricted portion of the total input to the network.

One example of a locally connected neural network is a convolutional neural network. FIG. 3C illustrates an example of a convolutional neural network **306**. The convolutional neural network **306** may be configured such that the connection strengths associated with the inputs for each neuron in the second layer are shared (e.g., **308**). Convolutional neural networks may be well suited to problems in which the spatial location of inputs is meaningful. Convolutional neural network **306** may be used to perform one or more aspects of video compression and/or decompression, according to aspects of the present disclosure.

One type of convolutional neural network is a deep convolutional network (DCN). FIG. 3D illustrates a detailed example of a DCN **300** designed to recognize visual features from an image **326** input from an image capturing device **330**, such as a car-mounted camera. The DCN **300** of the current example may be trained to identify traffic signs and a number provided on the traffic sign. Of course, the DCN **300** may be trained for other tasks, such as identifying lane markings or identifying traffic lights.

The DCN **300** may be trained with supervised learning. During training, the DCN **300** may be presented with an image, such as the image **326** of a speed limit sign, and a forward pass may then be computed to produce an output **322**. The DCN **300** may include a feature extraction section and a classification section. Upon receiving the image **326**, a convolutional layer **332** may apply convolutional kernels (not shown) to the image **326** to generate a first set of feature maps **318**. As an example, the convolutional kernel for the convolutional layer **332** may be a 5×5 kernel that generates 28×28 feature maps. In the present example, because four different feature maps are generated in the first set of feature maps **318**, four different convolutional kernels were applied to the image **326** at the convolutional layer **332**. The convolutional kernels may also be referred to as filters or convolutional filters.

The first set of feature maps **318** may be subsampled by a max pooling layer (not shown) to generate a second set of feature maps **320**. The max pooling layer reduces the size of the first set of feature maps **318**. That is, a size of the second set of feature maps **320**, such as 14×14, is less than the size of the first set of feature maps **318**, such as 28×28. The reduced size provides similar information to a subsequent layer while reducing memory consumption. The second set of feature maps **320** may be further convolved via one or more subsequent convolutional layers (not shown) to generate one or more subsequent sets of feature maps (not shown).

In the example of FIG. 3D, the second set of feature maps **320** is convolved to generate a first feature vector **324**. Furthermore, the first feature vector **324** is further convolved to generate a second feature vector **328**. Each feature of the second feature vector **328** may include a number that corresponds to a possible feature of the image **326**, such as “sign,” “60,” and “100.” A softmax function (not shown) may convert the numbers in the second feature vector **328** to

18

a probability. As such, an output **322** of the DCN **300** is a probability of the image **326** including one or more features.

In the present example, the probabilities in the output **322** for “sign” and “60” are higher than the probabilities of the others of the output **322**, such as “30,” “40,” “50,” “70,” “80,” “90,” and “100.” Before training, the output **322** produced by the DCN **300** is likely to be incorrect. Thus, an error may be calculated between the output **322** and a target output. The target output is the ground truth of the image **326** (e.g., “sign” and “60”). The weights of the DCN **300** may then be adjusted so the output **322** of the DCN **300** is more closely aligned with the target output.

To adjust the weights, a learning algorithm may compute a gradient vector for the weights. The gradient may indicate an amount that an error would increase or decrease if the weight were adjusted. At the top layer, the gradient may correspond directly to the value of a weight connecting an activated neuron in the penultimate layer and a neuron in the output layer. In lower layers, the gradient may depend on the value of the weights and on the computed error gradients of the higher layers. The weights may then be adjusted to reduce the error. This manner of adjusting the weights may be referred to as “back propagation” as it involves a “backward pass” through the neural network.

In practice, the error gradient of weights may be calculated over a small number of examples, so that the calculated gradient approximates the true error gradient. This approximation method may be referred to as stochastic gradient descent. Stochastic gradient descent may be repeated until the achievable error rate of the entire system has stopped decreasing or until the error rate has reached a target level. After learning, the DCN may be presented with new images and a forward pass through the network may yield an output **322** that may be considered an inference or a prediction of the DCN.

Deep belief networks (DBNs) are probabilistic models comprising multiple layers of hidden nodes. DBNs may be used to extract a hierarchical representation of training data sets. A DBN may be obtained by stacking up layers of Restricted Boltzmann Machines (RBMs). An RBM is a type of artificial neural network that can learn a probability distribution over a set of inputs. Because RBMs can learn a probability distribution in the absence of information about the class to which each input should be categorized, RBMs are often used in unsupervised learning. Using a hybrid unsupervised and supervised paradigm, the bottom RBMs of a DBN may be trained in an unsupervised manner and may serve as feature extractors, and the top RBM may be trained in a supervised manner (on a joint distribution of inputs from the previous layer and target classes) and may serve as a classifier.

Deep convolutional networks (DCNs) are networks of convolutional networks, configured with additional pooling and normalization layers. DCNs have achieved state-of-the-art performance on many tasks. DCNs can be trained using supervised learning in which both the input and output targets are known for many exemplars and are used to modify the weights of the network by use of gradient descent methods.

DCNs may be feed-forward networks. In addition, as described above, the connections from a neuron in a first layer of a DCN to a group of neurons in the next higher layer are shared across the neurons in the first layer. The feed-forward and shared connections of DCNs may be exploited for fast processing. The computational burden of a DCN

may be much less, for example, than that of a similarly sized neural network that comprises recurrent or feedback connections.

The processing of each layer of a convolutional network may be considered a spatially invariant template or basis projection. If the input is first decomposed into multiple channels, such as the red, green, and blue channels of a color image, then the convolutional network trained on that input may be considered three-dimensional, with two spatial dimensions along the axes of the image and a third dimension capturing color information. The outputs of the convolutional connections may be considered to form a feature map in the subsequent layer, with each element of the feature map (e.g., feature maps 320) receiving input from a range of neurons in the previous layer (e.g., feature maps 318) and from each of the multiple channels. The values in the feature map may be further processed with a non-linearity, such as a rectification, $\max(0, x)$. Values from adjacent neurons may be further pooled, which corresponds to down sampling, and may provide additional local invariance and dimensionality reduction.

FIG. 4 is a block diagram illustrating an example of a deep convolutional network 450. The deep convolutional network 450 may include multiple different types of layers based on connectivity and weight sharing. As shown in FIG. 4, the deep convolutional network 450 includes the convolution blocks 454A, 454B. Each of the convolution blocks 454A, 454B may be configured with a convolution layer (CONV) 456, a normalization layer (LNorm) 458, and a max pooling layer (MAX POOL) 460.

The convolution layers 456 may include one or more convolutional filters, which may be applied to the input data 452 to generate a feature map. Although only two convolution blocks 454A, 454B are shown, the present disclosure is not so limiting, and instead, any number of convolution blocks (e.g., convolution blocks 454A, 454B) may be included in the deep convolutional network 450 according to design preference. The normalization layer 458 may normalize the output of the convolution filters. For example, the normalization layer 458 may provide whitening or lateral inhibition. The max pooling layer 460 may provide down sampling aggregation over space for local invariance and dimensionality reduction.

The parallel filter banks, for example, of a deep convolutional network may be loaded on a CPU 212 or GPU 214 of the compute components 210 to achieve high performance and low power consumption. In alternative aspects, the parallel filter banks may be loaded on the DSP 216 or an ISP 218 of the compute components 210. In addition, the deep convolutional network 450 may access other processing blocks that may be present on the compute components 210, such as sensor processor and navigation module, dedicated, respectively, to sensors and navigation.

The deep convolutional network 450 may also include one or more fully connected layers, such as layer 462A (labeled "FC1") and layer 462B (labeled "FC2"). The deep convolutional network 450 may further include a logistic regression (LR) layer 464. Between each layer 456, 458, 460, 462A, 462B, 464 of the deep convolutional network 450 are weights (not shown) that are to be updated. The output of each of the layers (e.g., 456, 458, 460, 462A, 462B, 464) may serve as an input of a succeeding one of the layers (e.g., 456, 458, 460, 462A, 462B, 464) in the deep convolutional network 450 to learn hierarchical feature representations from input data 452 (e.g., images, audio, video, sensor data and/or other input data) supplied at the first of the convolution blocks 454A. The output of the deep

convolutional network 450 is a classification score 466 for the input data 452. The classification score 466 may be a set of probabilities, where each probability is the probability of the input data including a feature from a set of features.

As indicated above, a SLAM system may generate a map of an environment based on feature points. In some cases, the map may be stored as a mesh representation of the feature points. This mesh representation can be fairly memory and/or processor intensive to handle. For example, an XR system including a SLAM system may store information associated with numerous feature points and how the feature points relate to each other as a mesh. In some cases, it may be useful to represent certain objects using an edge-based representation.

FIG. 5 illustrates an edge-based representation of objects in an environment, in accordance with aspects of the present disclosure. In FIG. 5, includes an image 502 as well as a line representation (e.g., edge-based, or wireframe representation) 504 of objects in the image. As shown, the line representation 504 represents three-dimensional lines of objects as one or more edges. In some cases, the lines may outline each of the objects. For example, chair 506 in image 502 may be outlined as a chair representation 508 in the line representation 504. In some cases, the lines may also outline portions of the object. For example, chair representation 508 includes lines which represent various details of chair 506. In some cases, the lines provide structural information about objects in an environment and allow the objects to be mapped to a three-dimensional virtual space (e.g., map of the environment). In some cases, the lines of the chair representation may connect two points in the three-dimensional virtual space. In some cases, rather than representing, for example, chair 506 with a dense mesh of points, the chair 605 may be represented by a relatively small set of points and lines.

FIG. 6 is a block diagram illustrating a process 600 for generating an edge-based representation for cross-layer applications, in accordance with aspects of the present disclosure. In process 600, an image 602 may be input to a line representation ML model 604. In some cases, the image 602 may be part of a set of images, such as from a video stream. The line representation ML model 604 may be any ML model, such as a neural network, trained to generate a line representation model 606 of an environment from images of the environment. In some cases, this line representation model 606 may be built up from multiple images of the environment. In some cases, the line representation ML model 604 may detect lines and junctions within an image and infer vanishing points to estimate depth information.

In some cases, to help make the line representation model 606 more useful, objects represented in the line representation model 606 may be labeled with semantic meaning. For example, a line representation of a chair 608 may be labelled as such to allow higher level applications to more easily use the line representation. In some cases, the line representation ML model 604, or another ML model, not shown, may be trained to semantically segment, (e.g., label) one or more objects in the line representation model 606. In other cases, it may be useful to perform semantic segmentation 610 on the image 602 to generate a semantically segmented image 612. For example, any image semantic segmentation technique may be used to label objects (such as chair 614) in the semantically segmented image 612. In some cases, the image semantic segmentation may be performed by a ML semantic segmentation model. In some cases, only certain objects may be labelled in the semantically segmented

image 612. For example, an application may be capable of handling (or currently configured to handle) certain objects and these objects may be a subset of recognized objects. As another example, the application may just request to recognize flat surfaces to allow a virtual object to be set down, and every other object/feature may be recognized as background. Thus, in some cases, it may be more efficient to label objects from the subset of recognized objects and effectively ignore other objects not within the subset of recognized objects.

In some cases, the line representation ML model 604 may also generate a 2D line representation of the image 602 in addition to a predicted 3D line representation model 606. In some cases, the line representation ML model 604 may generate the 2D line representation of the image 602 as a part of identifying lines and junctions in image 602 prior to predicting depth information (e.g., projecting the 2D line representation into 3D). In some cases, based on the 2D line representation and the predicted 3D line representation model 606, a projection matrix 618 may be determined (or inferred or calculated). In some cases, the projection matrix 618 may indicate transformations applied to the lines and/or junctions used to project (e.g., back-project) from the 2D line representation to the 3D line representation model 606. In some cases, the line representation ML model 604 may output the projection matrix 618 directly. The 3D line representation model 606 along with the projection matrix 618 or 2D line representation may be input to a line representation semantic fusion engine 620.

In some cases, information from the semantically segmented image 612 may be fused with the 3D line representation model 606 based in part of the projection matrix 618 or the 2D line representation in combination with the 3D line representation model 606 by the line representation semantic fusion engine 620. In some cases, the line representation semantic fusion engine 620 may be a ML model trained to fuse information from the semantically segmented image 612 with the line representation model 606. In other cases, the line representation semantic fusion engine 620 may apply non-ML techniques, such as majority voting or other such fusion technique which may include, but are not limited to a weighted fusion strategy, a per-pixel, per-frame confidence score which learns based on a surface normal of current pixel in 3D and camera view angle, where a high weight may be assigned to a pixel if the score is large and a low weight assigned if the score is small, multiple-view frames may be combined via weighted fusion, reliable semantic label may be applied for each vertex, or other such fusion techniques. In some cases, the line representation semantic fusion engine 620 may apply a hybrid of ML and non-ML techniques.

As an example of operations that may be performed by the line representation semantic fusion engine 620, the semantically segmented image 612 may be applied to a two-dimensional projection of the 3D line representation model 606 and then back-projected into 3D space. L may represent the semantically segmented image 612 and $L(i,j)=C$ may represent a semantic label at location (i,j) of L , where C may be equal to one of set $[0, \dots, K]$, where the set of $[0, \dots, K]$ represents available labels and 0 of the set of $[0, \dots, K]$ represents a background. The 2D line representation may be represented by W , where $W(i,j)=B$ represents a status of a location of the 2D line representation where B may be one of set $[0, 1]$, where 0 indicates that a location is not a part of a line in the 2D line representation, and 1 indicates that the location is a part of a line in the 2D line representation. In cases where the 2D line representation is

provided by the line representation ML model 604, the 2D line representation may be directly used. In cases where a projection matrix 618 is provided along with the 3D line representation model 606, the 3D line representation model 606 may be projected to 2D to generate the 2D line representation based on the projection matrix 618. Labels from the semantically segmented image 612 may then be mapped to a line representation I using $I(i,j)=W(i,j)*L(i,j)$. Thus, locations not corresponding to a line in the 2D line representation are dropped and locations corresponding to a line may be kept along with the labels from the semantically segmented image 612. In some cases, labels for the background (e.g., 0 of the set of $[0, \dots, K]$ such that $W(i,j)=0$) may be ignored. Back-projection may be performed for $\Pi^{-1}(i,j)*I(i,j)$ to lift the labels from the 2D line representation, where $\Pi(i,j)$ represents the projection matrix 618 project 2D points from the 2D representation to the 3D line representation to determine labels for vertices of the 3D line representation. Output of the line representation semantic fusion engine 620 may be a semantic 3D representation 624 of objects in the environment that may be used to render as a view for display. The semantic 3D representation 624 of objects in the environment may be stored, for example in storage 207 of FIG. 2, so that additional images may provide additional views of an object and may be used to generate additional semantic 3D representations 624 that may be merged with the semantic 3D representation 624 generated based on the initial image.

In some cases, majority voting may be performed along edges of the line representation model 606 to map labels from the semantically segmented image 612 to the semantic 3D representations 624 across views. Generally, majority voting in ML systems may take a most common prediction from across multiple predictions, for example, by multiple classifiers, or multiple predictions from the same classifier, and use the most common prediction. For example, if a line is determined to belong to a part of the background in a first image, where the line appears in a second image, the lines may be matched and the line may be redetermined for the second image. If the line is determined to be a part of an object, the line may remain labelled as a part of the background. If the line appears in a third time and the line is determined to be part of an object in the third image, the line may be labelled as part of the object. In some cases, certain weights may be applied to a view of the line certain images, for example based on an angle of the line, a normal of the line, and the like. Majority voting may then be performed based on multiple weighted views of the line. The label which is determined the most may then be applied to the line.

In some cases, training for process 600 generally may be performed, at least in part, based on architectural drawings and/or computer aided design (CAD) drawings. For example, architectural drawings and/or CAD drawings are often created in a 3D wireframe and may include labels for certain objects in the 3D wireframe (e.g., for "stock" or template objects). Additionally, CAD and/or architectural drawing applications may be able to generate 3D wireframe environments and render these 3D wireframe environments as a rendered view. The labels generated by the CAD and/or architectural drawings may be used as a ground truth for training, for example, ML models for the line representation ML model 604, line representation semantic fusion engine 620, and/or process 600 as a whole. Input for training may be one or more views of the rendered environment by the CAD and/or architectural applications. Additionally, if the CAD and/or architectural drawing has been used to make a

physical object, images of the physical object may be input for training (or testing). A loss for training may be determined based on a difference between the labelled 3D wire-frame environments created by the CAD and/or architectural applications and the semantic 3D representation 624.

FIG. 7 is a flow diagram illustrating a process 700 for image processing, in accordance with aspects of the present disclosure. The process 700 may be performed by a computing device (or apparatus) or a component (e.g., a chipset, codec, etc.) of the computing device, such as host processor 152 of FIG. 1, compute components 210 of FIG. 2, and/or processor 1010 of FIG. 10. The computing device may be a mobile device (e.g., a mobile phone, mobile handset 950 of FIGS. 9A and 9B), a network-connected wearable such as a watch, an extended reality (XR) device such as a virtual reality (VR) device or augmented reality (AR) device (e.g., HMD 810 of FIGS. 8A and 8B, mobile handset 950 of FIGS. 9A and 9B), a vehicle or component or system of a vehicle, or other type of computing device. The operations of the process 700 may be implemented as software components that are executed and run on one or more processors (e.g., host processor 152 of FIG. 1, compute components 210 of FIG. 2, and/or processor 1010 of FIG. 10).

At block 702, the computing device (or component thereof) may obtain, from one or more image sensors (e.g., image sensor 130 of FIG. 1, image sensor 202 of FIG. 2, cameras 803 of FIGS. 8A and 8B, and/or cameras 930 of FIGS. 9A and 9B), a first image of an environment. In some cases, the first image is a first view of the object.

At block 704, the computing device (or component thereof) may determine semantic labels for a plurality of pixels of the first image based on whether each pixel of the plurality of pixels is associated with an object in the first image to generate a semantically segmented image.

At block 706, the computing device (or component thereof) may apply a two-dimensional line representation of the first image to the semantically segmented image to generate a labeled 2D line representation. The computing device (or component thereof) may generate the two-dimensional line representation of the first image. The computing device (or component thereof) may determine a projection matrix for projecting the line representation model to two dimensions. The computing device (or component thereof) may generate the two-dimensional line representation of the first image based on the projection matrix.

At block 708, the computing device (or component thereof) may back-project the labeled 2D line representation to three dimensions to generate a line representation model. In some cases, the line representation model comprises a three-dimensional model. In some cases, the labeled 2D line representation is back-projected based on the projection matrix.

At block 710, the computing device (or component thereof) may fuse the line representation model and the semantically segmented image to generate a labeled line representation model. In some cases, the fusing is performed by a machine learning model. The computing device (or component thereof) may receive a second image. In some cases, the second image is a second view of the object. The computing device (or component thereof) may generate a second labeled line representation model based on the second image. The computing device (or component thereof) may match a first line in the second labeled line representation model of the second image to a second line in the labeled line representation model. The computing device (or component thereof) may determine a semantic label for the matched first line and second line based on the first

image and second image. The computing device (or component thereof) may receive a set of object types. In some cases, the semantic labels for the plurality of pixels of the first image are determined based on the set of object types.

At block 712, the computing device (or component thereof) may output the labeled line representation model.

FIG. 8A is a perspective diagram 800 illustrating a head-mounted display (HMD) 810 that performs feature tracking and/or visual simultaneous localization and mapping (VSLAM), in accordance with some examples. The HMD 810 may be, for example, an augmented reality (AR) headset, a virtual reality (VR) headset, a mixed reality (MR) headset, an extended reality (XR) headset, or some combination thereof. The HMD 810 may be an example of an XR system 200, a SLAM system, or a combination thereof. The HMD 810 includes a first camera 830A and a second camera 830B along a front portion of the HMD 810. The first camera 830A and the second camera 830B may be two of the one or more cameras 38. In some examples, the HMD 810 may only have a single camera. In some examples, the HMD 810 may include one or more additional cameras in addition to the first camera 830A and the second camera 830B. In some examples, the HMD 810 may include one or more additional sensors in addition to the first camera 830A and the second camera 830B.

FIG. 8B is a perspective diagram 830 illustrating the head-mounted display (HMD) 810 of FIG. 8A being worn by a user 820, in accordance with some examples. The user 820 wears the HMD 810 on the user 820's head over the user 820's eyes. The HMD 810 can capture images with the first camera 830A and the second camera 830B. In some examples, the HMD 810 displays one or more display images toward the user 820's eyes that are based on the images captured by the first camera 830A and the second camera 830B. The display images may provide a stereoscopic view of the environment, in some cases with information overlaid and/or with other modifications. For example, the HMD 810 can display a first display image to the user 820's right eye, the first display image based on an image captured by the first camera 830A. The HMD 810 can display a second display image to the user 820's left eye, the second display image based on an image captured by the second camera 830B. For instance, the HMD 810 may provide overlaid information in the display images overlaid over the images captured by the first camera 830A and the second camera 830B.

The HMD 810 may include no wheels, propellers or other conveyance of its own. Instead, the HMD 810 relies on the movements of the user 820 to move the HMD 810 about the environment. In some cases, for instance where the HMD 810 is a VR headset, the environment may be entirely or partially virtual. If the environment is at least partially virtual, then movement through the virtual environment may be virtual as well. For instance, movement through the virtual environment can be controlled by an input device 208. The movement actuator may include any such input device 208. Movement through the virtual environment may not require wheels, propellers, legs, or any other form of conveyance. Even if an environment is virtual, SLAM techniques may still be valuable, as the virtual environment can be unmapped and/or may have been generated by a device other than the HMD 810, such as a remote server or console associated with a video game or video game platform. In some cases, feature tracking and/or SLAM may be performed in a virtual environment even by a vehicle or other device that has its own physical conveyance system that allows it to physically move about a physical environ-

25

ment. For example, SLAM may be performed in a virtual environment to test whether a SLAM system is working properly without wasting time or energy on movement and without wearing out a physical conveyance system.

FIG. 9A is a perspective diagram 900 illustrating a front surface 955 of a mobile device 950 that performs features described here, including, for example, feature tracking and/or visual simultaneous localization and mapping (VSLAM) using one or more front-facing cameras 930A-B, in accordance with some examples. The mobile device 950 may be, for example, a cellular telephone, a satellite phone, a portable gaming console, a music player, a health tracking device, a wearable device, a wireless communication device, a laptop, a mobile device, any other type of computing device or computing system 1300 discussed herein, or a combination thereof. The front surface 955 of the mobile device 950 includes a display screen 945. The front surface 955 of the mobile device 950 includes a first camera 930A and a second camera 930B. The first camera 930A and the second camera 930B are illustrated in a bezel around the display screen 945 on the front surface 955 of the mobile device 950. In some examples, the first camera 930A and the second camera 930B can be positioned in a notch or cutout that is cut out from the display screen 945 on the front surface 955 of the mobile device 950. In some examples, the first camera 930A and the second camera 930B can be under-display cameras that are positioned between the display screen 945 and the rest of the mobile device 950, so that light passes through a portion of the display screen 945 before reaching the first camera 930A and the second camera 930B. The first camera 930A and the second camera 930B of the perspective diagram 900 are front-facing cameras. The first camera 930A and the second camera 930B face a direction perpendicular to a planar surface of the front surface 955 of the mobile device 950. The first camera 930A and the second camera 930B may be two of the one or more cameras 310. In some examples, the front surface 955 of the mobile device 950 may only have a single camera. In some examples, the mobile device 950 may include one or more additional cameras in addition to the first camera 930A and the second camera 930B.

FIG. 9B is a perspective diagram 910 illustrating a rear surface 965 of a mobile device 950. The mobile device 950 includes a third camera 930C and a fourth camera 930D on the rear surface 965 of the mobile device 950. The third camera 930C and the fourth camera 930D of the perspective diagram 990 are rear-facing. The third camera 930C and the fourth camera 930D face a direction perpendicular to a planar surface of the rear surface 965 of the mobile device 950. While the rear surface 965 of the mobile device 950 does not have a display screen 945 as illustrated in the perspective diagram 990, in some examples, the rear surface 965 of the mobile device 950 may have a second display screen. If the rear surface 965 of the mobile device 950 has a display screen 945, any positioning of the third camera 930C and the fourth camera 930D relative to the display screen 945 may be used as discussed with respect to the first camera 930A and the second camera 930B at the front surface 955 of the mobile device 950. The third camera 930C and the fourth camera 930D may be two of the one or more cameras 39. In some examples, the rear surface 965 of the mobile device 950 may only have a single camera. In some examples, the mobile device 950 may include one or more additional cameras in addition to the first camera

26

930A, the second camera 930B, the third camera 930C, and the fourth camera 930D. In some examples, the mobile device 950 may include one or more additional sensors in addition to the first camera 930A, the second camera 930B, the third camera 930C, and the fourth camera 930D.

Like the HMD 810, the mobile device 950 includes no wheels, propellers, or other conveyance of its own. Instead, the mobile device 950 relies on the movements of a user holding or wearing the mobile device 950 to move the mobile device 950 about the environment. In some cases, for instance where the mobile device 950 is used for AR, VR, MR, or XR, the environment may be entirely or partially virtual. In some cases, the mobile device 950 may be slotted into a head-mounted device (HMD) (e.g., into a cradle of the HMD) so that the mobile device 950 functions as a display of the HMD, with the display screen 945 of the mobile device 950 functioning as the display of the HMD. If the environment is at least partially virtual, then movement through the virtual environment may be virtual as well. For instance, movement through the virtual environment can be controlled by one or more joysticks, buttons, video game controllers, mice, keyboards, trackpads, and/or other input devices that are coupled in a wired or wireless fashion to the mobile device 950.

FIG. 10 is a diagram illustrating an example of a system for implementing certain aspects of the present technology. In particular, FIG. 10 illustrates an example of computing system 1000, which can be for example any computing device making up internal computing system, a remote computing system, a camera, or any component thereof in which the components of the system are in communication with each other using connection 1005. Connection 1005 can be a physical connection using a bus, or a direct connection into processor 1010, such as in a chipset architecture. Connection 1005 can also be a virtual connection, networked connection, or logical connection.

In some examples, computing system 1000 is a distributed system in which the functions described in this disclosure can be distributed within a datacenter, multiple data centers, a peer network, etc. In some examples, one or more of the described system components represents many such components each performing some or all of the functions for which the component is described. In some cases, the components can be physical or virtual devices.

Example system 1000 includes at least one processing unit (CPU or processor) 1010 and connection 1005 that couples various system components including system memory 1015, such as read-only memory (ROM) 1020 and random access memory (RAM) 1025 to processor 1010. Computing system 1000 can include a cache 1012 of high-speed memory connected directly with, in close proximity to, or integrated as part of processor 1010.

Processor 1010 can include any general purpose processor and a hardware service or software service, such as services 1032, 1034, and 1036 stored in storage device 1030, configured to control processor 1010 as well as a special-purpose processor where software instructions are incorporated into the actual processor design. Processor 1010 may be a completely self-contained computing system, containing multiple cores or processors, a bus, memory controller, cache, etc. A multi-core processor may be symmetric or asymmetric.

To enable user interaction, computing system 1000 includes an input device 1045, which can represent any number of input mechanisms, such as a microphone for speech, a touch-sensitive screen for gesture or graphical input, keyboard, mouse, motion input, speech, camera,

accelerometers, gyroscopes, etc. Computing system **1000** can also include output device **1035**, which can be one or more of a number of output mechanisms. In some instances, multimodal systems can enable a user to provide multiple types of input/output to communicate with computing system **1000**. Computing system **1000** can include communications interface **1040**, which can generally govern and manage the user input and system output. The communication interface may perform or facilitate receipt and/or transmission of wired or wireless communications using wired and/or wireless transceivers, including those making use of an audio jack/plug, a microphone jack/plug, a universal serial bus (USB) port/plug, an Apple® Lightning® port/plug, an Ethernet port/plug, a fiber optic port/plug, a proprietary wired port/plug, a BLUETOOTH® wireless signal transfer, a BLUETOOTH® low energy (BLE) wireless signal transfer, an IBEACON® wireless signal transfer, a radio-frequency identification (RFID) wireless signal transfer, near-field communications (NFC) wireless signal transfer, dedicated short range communication (DSRC) wireless signal transfer, 802.10 Wi-Fi wireless signal transfer, wireless local area network (WLAN) signal transfer, Visible Light Communication (VLC), Worldwide Interoperability for Microwave Access (WiMAX), Infrared (IR) communication wireless signal transfer, Public Switched Telephone Network (PSTN) signal transfer, Integrated Services Digital Network (ISDN) signal transfer, 3G/4G/5G/LTE cellular data network wireless signal transfer, ad-hoc network signal transfer, radio wave signal transfer, microwave signal transfer, infrared signal transfer, visible light signal transfer, ultraviolet light signal transfer, wireless signal transfer along the electromagnetic spectrum, or some combination thereof. The communications interface **1040** may also include one or more Global Navigation Satellite System (GNSS) receivers or transceivers that are used to determine a location of the computing system **1000** based on receipt of one or more signals from one or more satellites associated with one or more GNSS systems. GNSS systems include, but are not limited to, the US-based Global Positioning System (GPS), the Russia-based Global Navigation Satellite System (GLO-NASS), the China-based BeiDou Navigation Satellite System (BDS), and the Europe-based Galileo GNSS. There is no restriction on operating on any particular hardware arrangement, and therefore the basic features here may easily be substituted for improved hardware or firmware arrangements as they are developed.

Storage device **1030** can be a non-volatile and/or non-transitory and/or computer-readable memory device and can be a hard disk or other types of computer readable media which can store data that are accessible by a computer, such as magnetic cassettes, flash memory cards, solid state memory devices, digital versatile disks, cartridges, a floppy disk, a flexible disk, a hard disk, magnetic tape, a magnetic strip/stripe, any other magnetic storage medium, flash memory, memristor memory, any other solid-state memory, a compact disc read only memory (CD-ROM) optical disc, a rewritable compact disc (CD) optical disc, digital video disk (DVD) optical disc, a blu-ray disc (BDD) optical disc, a holographic optical disk, another optical medium, a secure digital (SD) card, a micro secure digital (microSD) card, a Memory Stick® card, a smartcard chip, a EMV chip, a subscriber identity module (SIM) card, a mini/micro/nano/pico SIM card, another integrated circuit (IC) chip/card, random access memory (RAM), static RAM (SRAM), dynamic RAM (DRAM), read-only memory (ROM), programmable read-only memory (PROM), erasable programmable read-only memory (EPROM), electrically erasable

programmable read-only memory (EEPROM), flash EPROM (FLASH EPROM), cache memory (L1/L2/L3/L4/L5/L#), resistive random-access memory (RRAM/ReRAM), phase change memory (PCM), spin transfer torque RAM (STT-RAM), another memory chip or cartridge, and/or a combination thereof.

The storage device **1030** can include software services, servers, services, etc., that when the code that defines such software is executed by the processor **1010**, it causes the system to perform a function. In some examples, a hardware service that performs a particular function can include the software component stored in a computer-readable medium in connection with the necessary hardware components, such as processor **1010**, connection **1005**, output device **1035**, etc., to carry out the function.

As used herein, the term “computer-readable medium” includes, but is not limited to, portable or non-portable storage devices, optical storage devices, and various other mediums capable of storing, containing, or carrying instruction(s) and/or data. A computer-readable medium may include a non-transitory medium in which data can be stored and that does not include carrier waves and/or transitory electronic signals propagating wirelessly or over wired connections. Examples of a non-transitory medium may include, but are not limited to, a magnetic disk or tape, optical storage media such as compact disk (CD) or digital versatile disk (DVD), flash memory, memory or memory devices. A computer-readable medium may have stored thereon code and/or machine-executable instructions that may represent a procedure, a function, a subprogram, a program, a routine, a subroutine, a module, a software package, a class, or any combination of instructions, data structures, or program statements. A code segment may be coupled to another code segment or a hardware circuit by passing and/or receiving information, data, arguments, parameters, or memory contents. Information, arguments, parameters, data, etc. may be passed, forwarded, or transmitted using any suitable means including memory sharing, message passing, token passing, network transmission, or the like.

In some examples, the computer-readable storage devices, mediums, and memories can include a cable or wireless signal containing a bit stream and the like. However, when mentioned, non-transitory computer-readable storage media expressly exclude media such as energy, carrier signals, electromagnetic waves, and signals per se.

Specific details are provided in the description above to provide a thorough understanding of the examples provided herein. However, it will be understood by one of ordinary skill in the art that the examples may be practiced without these specific details. For clarity of explanation, in some instances the present technology may be presented as including individual functional blocks including functional blocks comprising devices, device components, steps or routines in a method embodied in software, or combinations of hardware and software. Additional components may be used other than those shown in the figures and/or described herein. For example, circuits, systems, networks, processes, and other components may be shown as components in block diagram form in order not to obscure the examples in unnecessary detail. In other instances, well-known circuits, processes, algorithms, structures, and techniques may be shown without unnecessary detail in order to avoid obscuring the examples.

Individual examples may be described above as a process or method which is depicted as a flowchart, a flow diagram, a data flow diagram, a structure diagram, or a block diagram.

Although a flowchart may describe the operations as a sequential process, many of the operations can be performed in parallel or concurrently. In addition, the order of the operations may be re-arranged. A process is terminated when its operations are completed, but could have additional steps not included in a figure. A process may correspond to a method, a function, a procedure, a subroutine, a subprogram, etc. When a process corresponds to a function, its termination can correspond to a return of the function to the calling function or the main function.

Processes and methods according to the above-described examples can be implemented using computer-executable instructions that are stored or otherwise available from computer-readable media. Such instructions can include, for example, instructions and data which cause or otherwise configure a general purpose computer, special purpose computer, or a processing device to perform a certain function or group of functions. Portions of computer resources used can be accessible over a network. The computer executable instructions may be, for example, binaries, intermediate format instructions such as assembly language, firmware, source code, etc. Examples of computer-readable media that may be used to store instructions, information used, and/or information created during methods according to described examples include magnetic or optical disks, flash memory, USB devices provided with non-volatile memory, networked storage devices, and so on.

Devices implementing processes and methods according to these disclosures can include hardware, software, firmware, middleware, microcode, hardware description languages, or any combination thereof, and can take any of a variety of form factors. When implemented in software, firmware, middleware, or microcode, the program code or code segments to perform the necessary tasks (e.g., a computer-program product) may be stored in a computer-readable or machine-readable medium. A processor(s) may perform the necessary tasks. Typical examples of form factors include laptops, smart phones, mobile phones, tablet devices or other small form factor personal computers, personal digital assistants, rackmount devices, standalone devices, and so on. Functionality described herein also can be embodied in peripherals or add-in cards. Such functionality can also be implemented on a circuit board among different chips or different processes executing in a single device, by way of further example.

The instructions, media for conveying such instructions, computing resources for executing them, and other structures for supporting such computing resources are example means for providing the functions described in the disclosure.

In the foregoing description, aspects of the application are described with reference to specific examples thereof, but those skilled in the art will recognize that the application is not limited thereto. Thus, while illustrative examples of the application have been described in detail herein, it is to be understood that the inventive concepts may be otherwise variously embodied and employed, and that the appended claims are intended to be construed to include such variations, except as limited by the prior art. Various features and aspects of the above-described application may be used individually or jointly. Further, examples can be utilized in any number of environments and applications beyond those described herein without departing from the broader spirit and scope of the specification. The specification and drawings are, accordingly, to be regarded as illustrative rather than restrictive. For the purposes of illustration, methods were described in a particular order. It should be appreciated

that in alternate examples, the methods may be performed in a different order than that described.

One of ordinary skill will appreciate that the less than (“<”) and greater than (“>”) symbols or terminology used herein can be replaced with less than or equal to (“≤”) and greater than or equal to (“≥”) symbols, respectively, without departing from the scope of this description.

Where components are described as being “configured to” perform certain operations, such configuration can be accomplished, for example, by designing electronic circuits or other hardware to perform the operation, by programming programmable electronic circuits (e.g., microprocessors, or other suitable electronic circuits) to perform the operation, or any combination thereof.

The phrase “coupled to” refers to any component that is physically connected to another component either directly or indirectly, and/or any component that is in communication with another component (e.g., connected to the other component over a wired or wireless connection, and/or other suitable communication interface) either directly or indirectly.

Claim language or other language reciting “at least one of” a set and/or “one or more” of a set indicates that one member of the set or multiple members of the set (in any combination) satisfy the claim. For example, claim language reciting “at least one of A and B” means A, B, or A and B. In another example, claim language reciting “at least one of A, B, and C” means A, B, C, or A and B, or A and C, or B and C, or A and B and C. The language “at least one of” a set and/or “one or more” of a set does not limit the set to the items listed in the set. For example, claim language reciting “at least one of A and B” can mean A, B, or A and B, and can additionally include items not listed in the set of A and B.

The various illustrative logical blocks, modules, circuits, and algorithm steps described in connection with the examples disclosed herein may be implemented as electronic hardware, computer software, firmware, or combinations thereof. To clearly illustrate this interchangeability of hardware and software, various illustrative components, blocks, modules, circuits, and steps have been described above generally in terms of their functionality. Whether such functionality is implemented as hardware or software depends upon the particular application and design constraints imposed on the overall system. Skilled artisans may implement the described functionality in varying ways for each particular application, but such implementation decisions should not be interpreted as causing a departure from the scope of the present application.

The techniques described herein may also be implemented in electronic hardware, computer software, firmware, or any combination thereof. Such techniques may be implemented in any of a variety of devices such as general purposes computers, wireless communication device handsets, or integrated circuit devices having multiple uses including application in wireless communication device handsets and other devices. Any features described as modules or components may be implemented together in an integrated logic device or separately as discrete but interoperable logic devices. If implemented in software, the techniques may be realized at least in part by a computer-readable data storage medium comprising program code including instructions that, when executed, performs one or more of the methods described above. The computer-readable data storage medium may form part of a computer program product, which may include packaging materials. The computer-readable medium may comprise memory or

31

data storage media, such as random access memory (RAM) such as synchronous dynamic random access memory (SDRAM), read-only memory (ROM), non-volatile random access memory (NVRAM), electrically erasable programmable read-only memory (EEPROM), FLASH memory, magnetic or optical data storage media, and the like. The techniques additionally, or alternatively, may be realized at least in part by a computer-readable communication medium that carries or communicates program code in the form of instructions or data structures and that can be accessed, read, and/or executed by a computer, such as propagated signals or waves.

The program code may be executed by a processor, which may include one or more processors, such as one or more digital signal processors (DSPs), general purpose microprocessors, an application specific integrated circuits (ASICs), field programmable logic arrays (FPGAs), or other equivalent integrated or discrete logic circuitry. Such a processor may be configured to perform any of the techniques described in this disclosure. A general purpose processor may be a microprocessor; but in the alternative, the processor may be any conventional processor, controller, microcontroller, or state machine. A processor may also be implemented as a combination of computing devices, e.g., a combination of a DSP and a microprocessor, a plurality of microprocessors, one or more microprocessors in conjunction with a DSP core, or any other such configuration. Accordingly, the term "processor," as used herein may refer to any of the foregoing structure, any combination of the foregoing structure, or any other structure or apparatus suitable for implementation of the techniques described herein. In addition, in some aspects, the functionality described herein may be provided within dedicated software modules or hardware modules configured for encoding and decoding, or incorporated in a combined video encoder-decoder (CODEC).

Illustrative aspects of the present disclosure include:

Aspect 1. A method for image processing, comprising: obtaining, from one or more image sensors, a first image of an environment; determining semantic labels for a plurality of pixels of the first image based on whether each pixel of the plurality of pixels is associated with an object in the first image to generate a semantically segmented image; applying a two-dimensional line representation of the first image to the semantically segmented image to generate a labeled 2D line representation; back-projecting the labeled 2D line representation to three dimensions to generate a line representation model; fusing the line representation model and the semantically segmented image to generate a labeled line representation model; and outputting the labeled line representation model.

Aspect 2. The method of Aspect 1, wherein the line representation model comprises a three-dimensional model.

Aspect 3. The method of any of Aspects 1-2, further comprising generating the two-dimensional line representation of the first image.

Aspect 4. The method of any of Aspects 1-3, further comprising: determining a projection matrix for projecting the line representation model to two dimensions; and generating the two-dimensional line representation of the first image based on the projection matrix.

Aspect 5. The method of Aspect 4, wherein the back-projecting is based on the projection matrix.

Aspect 6. The method of any of Aspects 1-5, wherein the first image is a first view of the object, and further comprising: receiving a second image, wherein the second image is a second view of the object; generating a second labeled

32

line representation model based on the second image; and matching a first line in the second labeled line representation model of the second image to a second line in the labeled line representation model.

Aspect 7. The method of Aspect 6, further comprising determining a semantic label for the matched first line and second line based on the first image and second image.

Aspect 8. The method of any of Aspects 1-7, wherein the fusing is performed by a machine learning model.

Aspect 9. The method of any of Aspects 1-8, further comprising receiving a set of object types, and wherein determining the semantic labels for the plurality of pixels of the first image is based on the set of object types.

Aspect 10. An apparatus for image processing, comprising: a memory comprising instructions; and at least one processor coupled to the memory and configured to: obtain, from one or more image sensors, a first image of an environment; determine semantic labels for a plurality of pixels of the first image based on whether each pixel of the plurality of pixels is associated with an object in the first image to generate a semantically segmented image; apply a two-dimensional line representation of the first image to the semantically segmented image to generate a labeled 2D line representation; back-project the labeled 2D line representation to three dimensions to generate a line representation model; fuse the line representation model and the semantically segmented image to generate a labeled line representation model; and output the labeled line representation model.

Aspect 11. The apparatus of Aspect 10, wherein the line representation model comprises a three-dimensional model.

Aspect 12. The apparatus of any of Aspects 10-11, wherein the at least one processor is further configured to generate the two-dimensional line representation of the first image.

Aspect 13. The apparatus of any of Aspects 10-12, wherein the at least one processor is further configured to: determine a projection matrix for projecting the line representation model to two dimensions; and generate the two-dimensional line representation of the first image based on the projection matrix.

Aspect 14. The apparatus of Aspect 13, wherein the labeled 2D line representation is back-projected based on the projection matrix.

Aspect 15. The apparatus of any of Aspects 10-14, wherein the fusing is performed by a machine learning model.

Aspect 16. The apparatus of any of Aspects 10-15, wherein the first image is a first view of the object, and wherein the at least one processor is further configured to: receive a second image, wherein the second image is a second view of the object; generate a second labeled line representation model based on the second image; and match a first line in the second labeled line representation model of the second image to a second line in the labeled line representation model.

Aspect 17. The apparatus of Aspect 16, wherein the at least one processor is further configured to determine a semantic label for the matched first line and second line based on the first image and second image.

Aspect 18. The apparatus of any of Aspects 10-17, wherein the at least one processor is further configured to receive a set of object types, and wherein the semantic labels for the plurality of pixels of the first image are determined based on the set of object types.

Aspect 19. A non-transitory computer-readable medium having stored thereon instructions that, when executed by at

33

least one processor, cause the at least one processor to: obtain, from one or more image sensors, a first image of an environment; determine semantic labels for a plurality of pixels of the first image based on whether each pixel of the plurality of pixels is associated with an object in the first image to generate a semantically segmented image; apply a two-dimensional line representation of the first image to the semantically segmented image to generate a labeled 2D line representation; back-project the labeled 2D line representation to three dimensions to generate a line representation model; fuse the line representation model and the semantically segmented image to generate a labeled line representation model; and output the labeled line representation model.

Aspect 20. The non-transitory computer-readable medium of Aspect 19, wherein the line representation model comprises a three-dimensional model.

Aspect 21. The non-transitory computer-readable medium of any of Aspects 19-20, wherein the instructions cause the at least one processor to generate the two-dimensional line representation of the first image.

Aspect 22. The non-transitory computer-readable medium of any of Aspects 19-21, wherein the instructions cause the at least one processor to: determine a projection matrix for projecting the line representation model to two dimensions; and generate the two-dimensional line representation of the first image based on the projection matrix.

Aspect 23. The non-transitory computer-readable medium of Aspect 22, wherein the labeled 2D line representation is back-projected based on the projection matrix.

Aspect 24. The non-transitory computer-readable medium of any of Aspects 19-23, wherein the fusing is performed by a machine learning model.

Aspect 25. The non-transitory computer-readable medium of any of Aspects 19-24, wherein the first image is a first view of the object, and wherein the instructions cause the at least one processor to: receive a second image, wherein the second image is a second view of the object; generate a second labeled line representation model based on the second image; and match a first line in the second labeled line representation model of the second image to a second line in the labeled line representation model.

Aspect 26. The non-transitory computer-readable medium of Aspect 25, wherein the instructions cause the at least one processor to determine a semantic label for the matched first line and second line based on the first image and second image.

Aspect 27. The non-transitory computer-readable medium of any of Aspects 19-26, wherein the instructions cause the at least one processor to receive a set of object types, and wherein the semantic labels for the plurality of pixels of the first image are determined based on the set of object types.

Aspect 28. An apparatus for image processing, comprising: means for obtaining, from one or more image sensors, a first image of an environment; means for determining semantic labels for a plurality of pixels of the first image based on whether each pixel of the plurality of pixels is associated with an object in the first image to generate a semantically segmented image; means for applying a two-dimensional line representation of the first image to the semantically segmented image to generate a labeled 2D line representation; means for back-projecting the labeled 2D line representation to three dimensions to generate a line representation model; means for fusing the line representation model and the semantically segmented image to generate a labeled line representation model; and means for outputting the labeled line representation model.

34

Aspect 29. The apparatus of Aspect 28, wherein the line representation model comprises a three-dimensional model.

Aspect 30. The apparatus of any of Aspects 28-29, further comprising means for generating the two-dimensional line representation of the first image.

Aspect 34: An apparatus for image generation, comprising means for performing one or more of operations according to any of Aspects 1 to 9.

What is claimed is:

1. A method for image processing, comprising: obtaining, from one or more image sensors, a first image of an environment; determining semantic labels for a plurality of pixels of the first image based on whether each pixel of the plurality of pixels is associated with an object in the first image to generate a semantically segmented image; applying a two-dimensional line representation of the first image to the semantically segmented image to generate a labeled 2D line representation; back-projecting the labeled 2D line representation to three dimensions to generate a line representation model; fusing the line representation model and the semantically segmented image to generate a labeled line representation model; and outputting the labeled line representation model.
2. The method of claim 1, wherein the line representation model comprises a three-dimensional model.
3. The method of claim 1, further comprising generating the two-dimensional line representation of the first image.
4. The method of claim 1, further comprising: determining a projection matrix for projecting the line representation model to two dimensions; and generating the two-dimensional line representation of the first image based on the projection matrix.
5. The method of claim 4, wherein the back-projecting is based on the projection matrix.
6. The method of claim 1, wherein the first image is a first view of the object, and further comprising: receiving a second image, wherein the second image is a second view of the object; generating a second labeled line representation model based on the second image; and matching a first line in the second labeled line representation model of the second image to a second line in the labeled line representation model.
7. The method of claim 6, further comprising determining a semantic label for the matched first line and second line based on the first image and second image.
8. The method of claim 1, wherein the fusing is performed by a machine learning model.
9. The method of claim 1, further comprising receiving a set of object types, and wherein determining the semantic labels for the plurality of pixels of the first image is based on the set of object types.
10. An apparatus for image processing, comprising: at least one memory comprising instructions; and at least one processor coupled to the at least one memory and configured to: obtain, from one or more image sensors, a first image of an environment; determine semantic labels for a plurality of pixels of the first image based on whether each pixel of the plurality of pixels is associated with an object in the first image to generate a semantically segmented image;

35

apply a two-dimensional line representation of the first image to the semantically segmented image to generate a labeled 2D line representation;
back-project the labeled 2D line representation to three dimensions to generate a line representation model;
fuse the line representation model and the semantically segmented image to generate a labeled line representation model; and
output the labeled line representation model.

11. The apparatus of claim 10, wherein the line representation model comprises a three-dimensional model.

12. The apparatus of claim 10, wherein the at least one processor is further configured to generate the two-dimensional line representation of the first image.

13. The apparatus of claim 10, wherein the at least one processor is further configured to:

determine a projection matrix for projecting the line representation model to two dimensions; and
generate the two-dimensional line representation of the first image based on the projection matrix.

14. The apparatus of claim 13, wherein the labeled 2D line representation is back-projected based on the projection matrix.

15. The apparatus of claim 10, wherein the at least one processor is configured to perform the fusing using a machine learning model.

16. The apparatus of claim 10, wherein the first image is a first view of the object, and wherein the at least one processor is further configured to:

receive a second image, wherein the second image is a second view of the object;
generate a second labeled line representation model based on the second image; and
match a first line in the second labeled line representation model of the second image to a second line in the labeled line representation model.

17. The apparatus of claim 16, wherein the at least one processor is further configured to determine a semantic label for the matched first line and second line based on the first image and second image.

18. The apparatus of claim 10, wherein the at least one processor is further configured to receive a set of object types, and wherein the semantic labels for the plurality of pixels of the first image are determined based on the set of object types.

19. A non-transitory computer-readable medium having stored thereon instructions that, when executed by at least one processor, cause the at least one processor to:

obtain, from one or more image sensors, a first image of an environment;

determine semantic labels for a plurality of pixels of the first image based on whether each pixel of the plurality of pixels is associated with an object in the first image to generate a semantically segmented image;

apply a two-dimensional line representation of the first image to the semantically segmented image to generate a labeled 2D line representation;

back-project the labeled 2D line representation to three dimensions to generate a line representation model;

fuse the line representation model and the semantically segmented image to generate a labeled line representation model; and

output the labeled line representation model.

20. The non-transitory computer-readable medium of claim 19, wherein the line representation model comprises a three-dimensional model.

36

21. The non-transitory computer-readable medium of claim 19, wherein the instructions cause the at least one processor to generate the two-dimensional line representation of the first image.

22. The non-transitory computer-readable medium of claim 19, wherein the instructions cause the at least one processor to:

determine a projection matrix for projecting the line representation model to two dimensions; and

generate the two-dimensional line representation of the first image based on the projection matrix.

23. The non-transitory computer-readable medium of claim 22, wherein the labeled 2D line representation is back-projected based on the projection matrix.

24. The non-transitory computer-readable medium of claim 19, wherein the fusing is performed by a machine learning model.

25. The non-transitory computer-readable medium of claim 19, wherein the first image is a first view of the object, and wherein the instructions cause the at least one processor to:

receive a second image, wherein the second image is a second view of the object;

generate a second labeled line representation model based on the second image; and

match a first line in the second labeled line representation model of the second image to a second line in the labeled line representation model.

26. The non-transitory computer-readable medium of claim 25, wherein the instructions cause the at least one processor to determine a semantic label for the matched first line and second line based on the first image and second image.

27. The non-transitory computer-readable medium of claim 19, wherein the instructions cause the at least one processor to receive a set of object types, and wherein the semantic labels for the plurality of pixels of the first image are determined based on the set of object types.

28. An apparatus for image processing, comprising:

means for obtaining, from one or more image sensors, a first image of an environment;

means for determining semantic labels for a plurality of pixels of the first image based on whether each pixel of the plurality of pixels is associated with an object in the first image to generate a semantically segmented image;

means for applying a two-dimensional line representation of the first image to the semantically segmented image to generate a labeled 2D line representation;

means for back-projecting the labeled 2D line representation to three dimensions to generate a line representation model;

means for fusing the line representation model and the semantically segmented image to generate a labeled line representation model; and

means for outputting the labeled line representation model.

29. The apparatus of claim 28, wherein the line representation model comprises a three-dimensional model.

30. The apparatus of claim 28, further comprising means for generating the two-dimensional line representation of the first image.

* * * * *